Computer Organization and Architecture (CSE1003)

Syllabus

Module I: Introduction to Computer Architecture & Data Representation

Module II: Processor Architecture & Instruction-Level Parallelism

Module III: Memory Systems & Cache Organization

Module IV: I/O Systems

Module V: Multiprocessors and Parallel Processing



MODULE I:

INTRODUCTION TO COMPUTER ARCHITECTURE & DATA REPRESENTATION

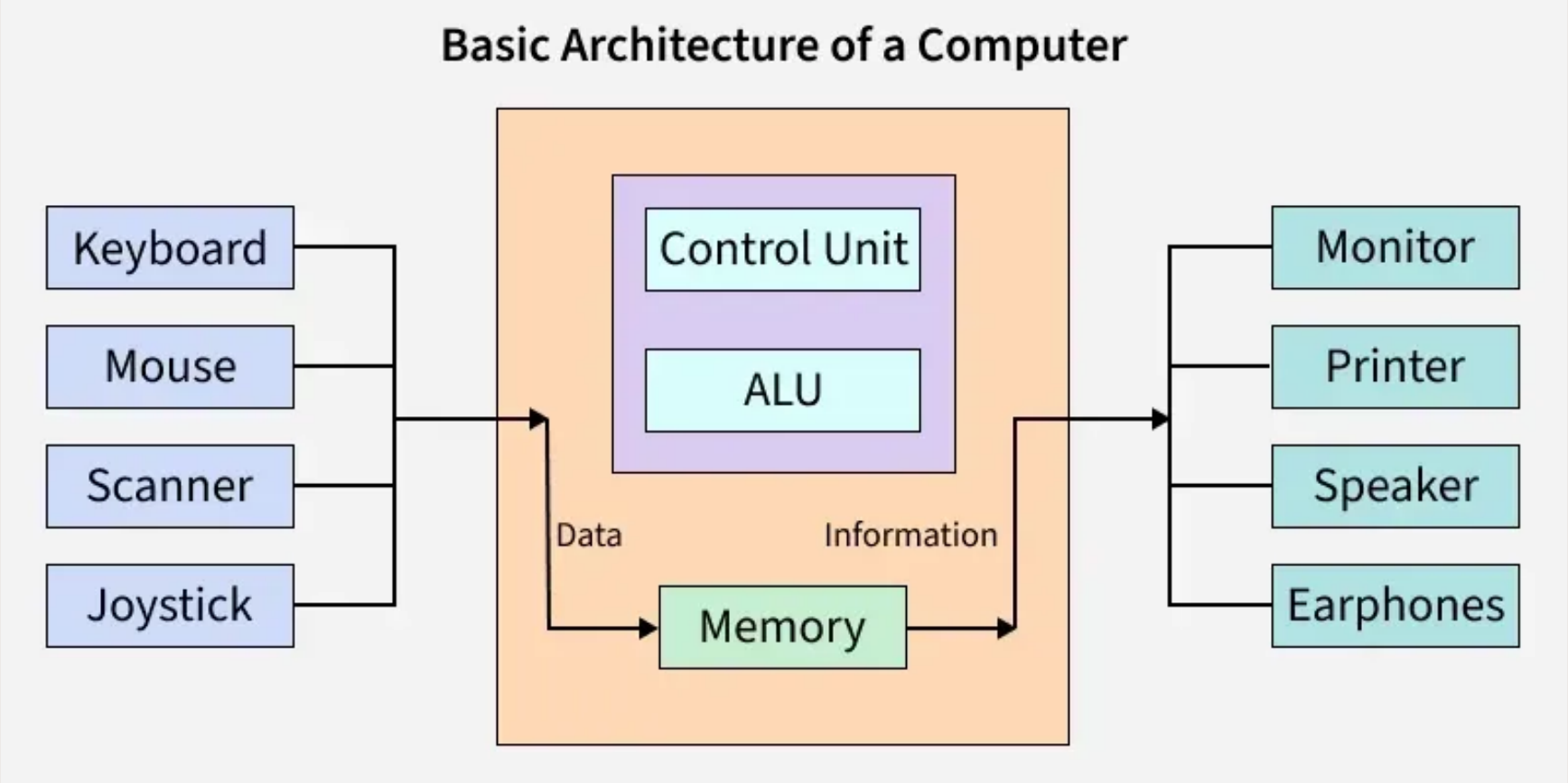
Prepared by- Ashwini Dhumal

Module I: Introduction to Computer Architecture & Data Representation

- **Introduction to Computer Architecture :**
- Evolution and history of computation,
- Computer architecture vs computer organization,
- Eight great ideas in computer architecture.
- **Basic Computer Organization:**
- Functional units of a computer,
- Block diagram of a computer system,
- CPU,
- memory.
- Programs,
- OS, and Hardware Interaction:
- Role of programs and compilers,
- Operating system interaction with hardware,
- Basic instruction execution concept.
- Number system.

INTRODUCTION TO COMPUTER

Computer is an electronic device that processes data according to instructions provided by software programs. It takes input (data), processes it using a central processing unit (CPU), stores information, and produces output (results) to perform various tasks. It is widely used in fields such as education, business, communication, and entertainment for efficient and accurate operations.



CHARACTERISTICS OF COMPUTER

1. Speed: Executing mathematical calculation, a computer works faster and more accurately than human. Computers have the ability to process so many millions (1,000,000) of instructions per second. Computer operations are performed in micro and nano seconds. The speed of a computer is measure in terms of GigaHertz and MegaHertz.

2. Diligence: A human cannot work for several hours without resting, yet a computer never tires. A computer can conduct millions of calculations per second with complete precision without stopping. A computer can consistently and accurately do millions of jobs or calculations.

3. Reliability

A computer is reliable. The output results never differ unless the input varies. the output is totally depend on the input. when an input is the same the output will also be the same. A computer produces consistent results for similar sets of data, if we provide the same set of input at any time we will get the same result.

4. Automation

The world is quickly moving toward **AI (Artificial Intelligence)**-based technology. A computer may conduct tasks automatically after instructions are programmed. By executing jobs automatically, this computer feature replaces thousands of workers.

5. Versatility

Versatility refers to a capacity of computer. Computer perform different types of tasks with the same accuracy and efficiency. A computer can perform multiple tasks at the same time this is known as versatility.

For example, while listening to music, we may develop our project using PowerPoint and Wordpad, or we can design a website.

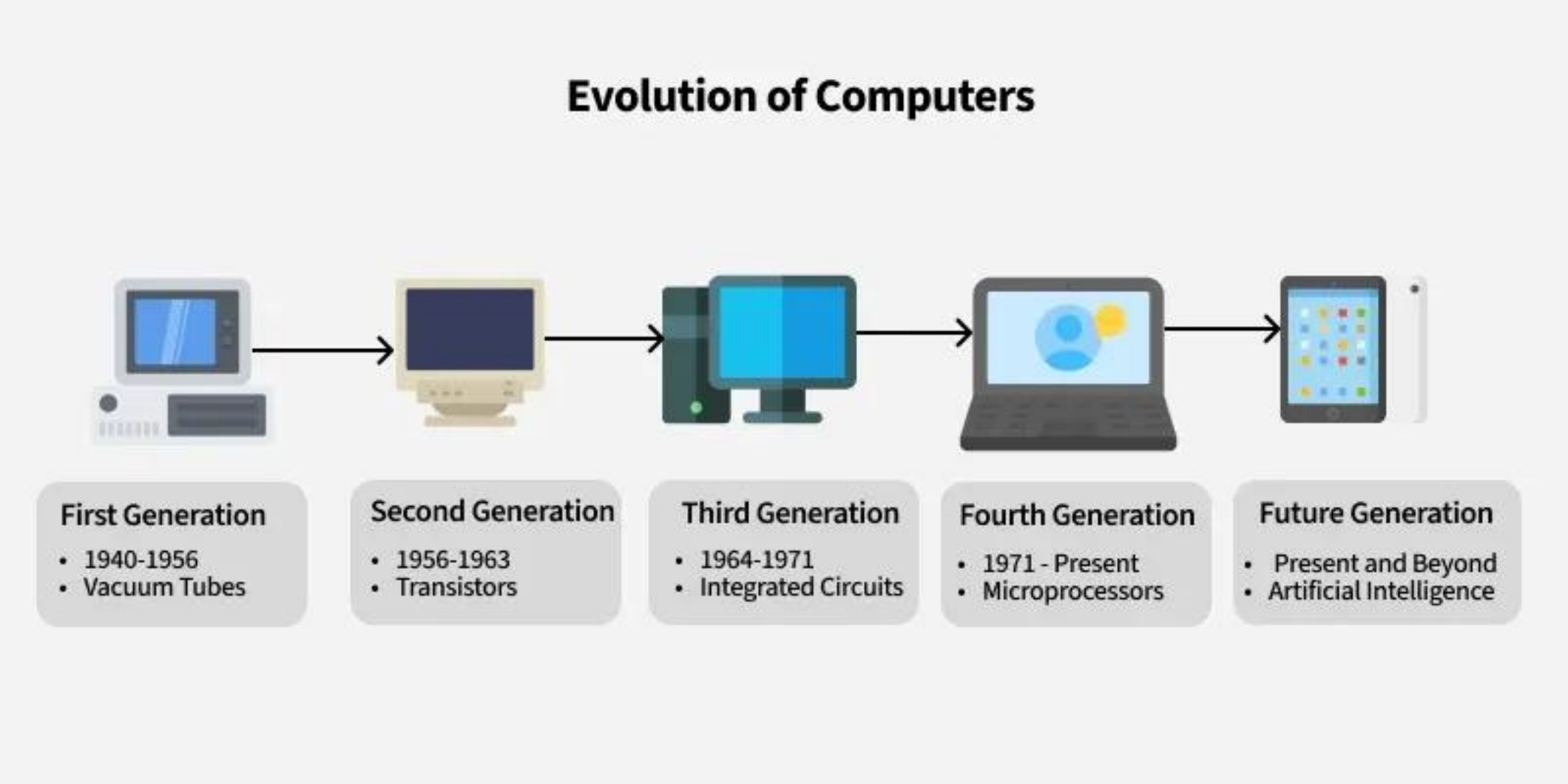
6. Memory

A computer can store millions of records and these records may be accessed with complete precision. Computer memory storage capacity is measured in Bytes, Kilobytes(KB), Megabytes(MB), Gigabytes(GB), and Terabytes(TB). A computer has built-in memory known as primary memory.

7. Accuracy

When a computer performs a computation or operation, the chances of errors occurring are low. Errors in a computer are caused by human's submitting incorrect data. A computer can do a variety of operations and calculations fast and accurately.

EVOLUTION OF COMPUTERS



1. First Generation (1940-1956): Vacuum Tubes

The first generation of computers relied on vacuum tubes, which were used for both circuitry and memory storage. These early machines were massive and primarily used for scientific and military applications.

Key Features:

- **Size**: Computers were enormous, often filling entire rooms.
- **Speed**: Processing speed was relatively slow due to the limitations of vacuum tubes.
- **Power Consumption**: These computers consumed large amounts of electrical power, leading to high operating costs.

Examples:
ENIAC (Electronic Numerical Integrator and Computer): One of the earliest general-purpose electronic computers.
UNIVAC (Universal Automatic Computer): The first commercially successful computer.

2. Second Generation (1956-1963): Transistors

The second generation saw the replacement of vacuum tubes with transistors, which were smaller, more reliable, and consumed less power. Transistors made computers more compact, efficient, and affordable.

Key Features:

- **Size**: Transistor-based computers were much smaller than their predecessors.
- **Speed**: Processing speeds improved, making computers more practical for commercial and research purposes.
- **Reliability**: Transistors were less prone to failure than vacuum tubes, improving the overall reliability of computers.

Examples:

- **IBM 7090**: A highly successful transistorized computer used in scientific and business applications.
- **CDC 1604**: One of the first computers to use transistors, used for scientific calculations.

3. Third Generation (1964-1971): Integrated Circuits

The third generation of computers introduced integrated circuits (ICs), where multiple transistors were integrated onto a single chip. This reduced the size of computers further, while increasing processing power and efficiency.

Key Features:

- **Size**: Integrated circuits allowed for more compact and powerful machines.
- **Speed**: Computers became faster, able to process data more efficiently.
- **Heat Generation**: ICs produced less heat than individual transistors, leading to improved system stability and performance.

Examples:

- **IBM System/360**: A family of computers that demonstrated the versatility of integrated circuits in various industries.
- **PDP-8**: One of the first commercially successful minicomputers, marking a shift towards more affordable computing.

4. Fourth Generation (1971-Present): Microprocessors

The fourth generation introduced the microprocessor, which combined all the components of a computer's central processing unit (CPU) onto a single chip. This innovation drastically reduced the size and cost of computers, leading to the rise of personal computers.

Key Features:

- **Size**: Computers became smaller and more affordable, allowing individuals to own personal computers.
- **Power**: Microprocessors made computers significantly more powerful, capable of handling more complex tasks.
- **User Interfaces**: The development of graphical user interfaces (GUIs) made computers more user-friendly and accessible to non-experts.

Examples:
Intel 4004: The first commercially available microprocessor, enabling the creation of personal computers.
Apple Macintosh: A popular personal computer that brought GUIs to a wider audience.

5. Fifth Generation (Present and Beyond): Artificial Intelligence

The fifth generation of computers is characterized by advancements in artificial intelligence (AI), machine learning, and quantum computing. These technologies enable computers to learn, reason, and process data in ways similar to human cognition.

Key Features:

- **Artificial Intelligence**: AI systems are capable of performing complex tasks such as natural language processing, image recognition, and autonomous decision-making.

- **Automation**: The development of smart systems and robots that can perform tasks autonomously in fields like healthcare, manufacturing, and logistics.

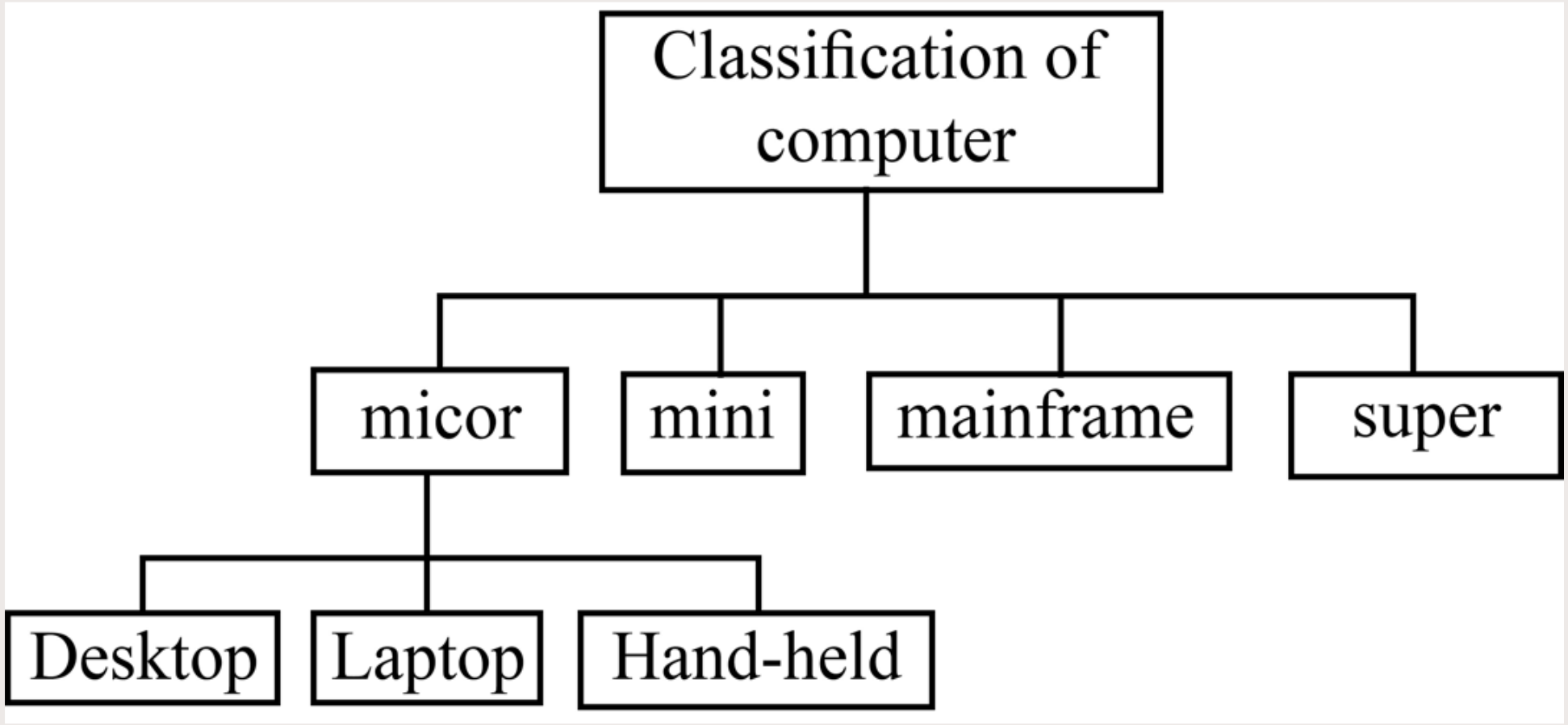
Examples:

- **IBM Watson**: An AI system that can analyze large datasets and answer questions posed in natural language.

- **Quantum Computers**: Still in the experimental phase, these have the potential to revolutionize fields such as cryptography and drug discovery.

CLASSIFICATION OF COMPUTERS

Four major categories: micro, mini, mainframe and super computers



Micro Computer

Microcomputers, also known as personal computers (PCs), are a type of computer designed for individual use. They are distinguished by their compact dimensions, small size, processing power, compatibility, internet connectivity, portability, low price, and versatility. In the 1970s and 1980s, microcomputers gained popularity and became more popular in the modern computing era.



Fig: Micro Computer

- **Size** – Microcomputers are small in size. These are portable.
- **Example** – Some of the popular microcomputers are laptops and desktops, standard PCs, mobile phones, and notebooks.

Minicomputer

A minicomputer is a type of computer that is smaller in size than large computers. It possesses all the capabilities of a large computer. Hence, it is a midsize multi-processing system capable of supporting up to 250 users simultaneously.



Fig: Mini Computer

- **Size** – Its size falls between mainframes and microcomputers. It is larger than mainframe computers and smaller than microcomputers.

Example – Some of the popular minicomputers are the PDP-11, IBM's AS/400e, Honeywell 200, and TI-990.

Mainframe computer

The mainframe is very large and is an expensive computer capable of supporting hundreds or even thousands of users simultaneously. The mainframe executes many programmes concurrently and supports simultaneous execution of programmes.



Fig: Mainframe Computer

Size – Mainframe computers can vary in size; their size generally depends on their specifications and the specific model being considered.

Example – Some of the popular mainframe computers are IBM zSeries mainframes (BM z14 and IBM z15), Unisys ClearPath, Fujitsu GS21 Series, and Hitachi VOS3.

Supercomputer

A supercomputer is a special type of computer that is more powerful and capable of high-performance computing. It is specifically designed to compute complex and intensive tasks that regular computers cannot do efficiently.

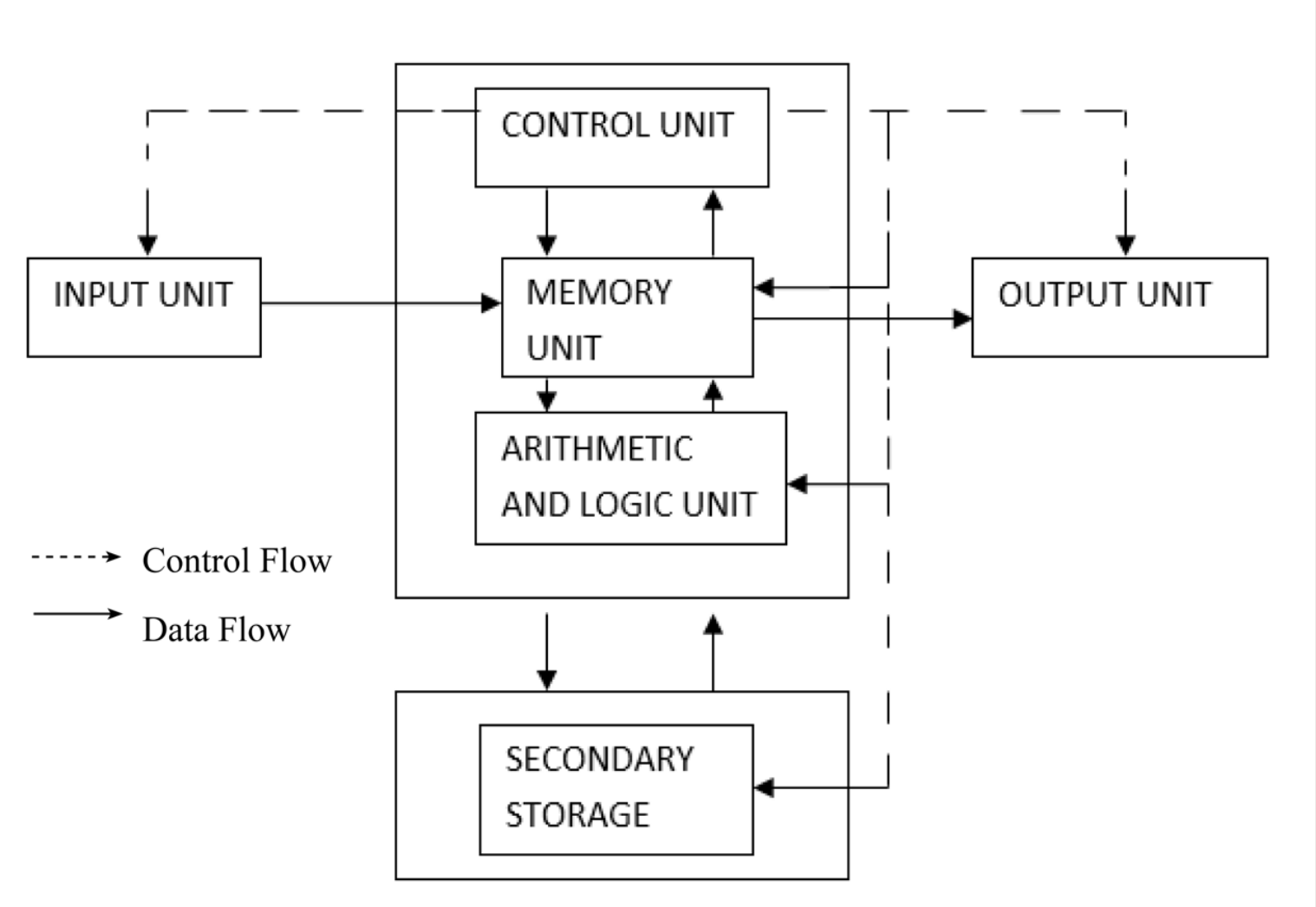
- **Size** – Supercomputers can vary in size, from small clusters of computers to massive installations. A supercomputer may contain 10, 100, 1000, or more computers that all work together.



Fig: Super Computer

- **Example** – Some of the popular supercomputers are Fugaku, Google Sycamore, Baidu's quantum supercomputer, and Sierra.

Introduction to Computer Architecture:



The block diagram of the computer system have the following three units, each functional unit corresponds to their basic operations performed as described in details.

(a) Input unit

(b) Central processing unit

(c) Output unit

a) Input Unit

- Accept data and instructions from the outside world.
- Convert it to a form that the computer can understand
- Supply the converted data to the computer system for further processing
- The input unit is used to send information or instructions or commands to the computer. The data received from the input unit is immediately stored in main memory and then processed.
- Following are the some of the input devise.

  (i) Keyboard

  (ii) Mouse

  (iii) Light pen

  (iv) Joystick

  (v) Ocr (optical character recognizer)

  (vi) MICR (magnetic ink character recognizer)

  (vii) OMR (optical mark recognizer)

(b) Central Processing Unit (CPU)

- It performs all calculations and all decisions.
- It controls and co-ordinates all units of the computer
- It interprets instructions of a program
- The CPU is sub-divided into the following sub-system.

  (i) Control unit

  (ii) Arithmetic and logical unit

  (iii) Memory unit

    (a) Primary storage

    (b) Secondary storage.

(i) Control unit

- The control unit instructs the computer how to carry out program instructions. It directs the flow of data between memory and arithmetic logical unit.
- The input unit does not know when to receive data and where to put the data in the storage unit after receiving it similarly, the control unit instructs the input unit where to store the data after receiving it from the user.
- In the same way, it controls the flow of data and instructions from the storage unit to ALU during program execution
- The control unit fetches instructions from the primary memory, decodes them to determine the operations required, and then sets up instruction's execution.
- ***Eg.*** To add two numbers or to read a character from a keyboard. A number registers are associated with the control unit.

(ii) Arithmetic And Logical Unit

- Arithmetic and logical unit performs all the arithmetic and logical operations. Arithmetic operations like addition, subtraction, multiplication and logical operations, such as comparisons are performed in ALU.
- ***Example:*** it can check if the number A is less than equal to or greater than the number B. once the calculations or the logical operation is performed by ALU, then the result is transferred to the storage unit.

(iii) Memory unit

- Memory is the part of computer which holds data for processing and other information it is also called as ***main memory*** or ***primary memory***.
- A device that stores program instructions or data used by the cpu when performing a given function.
- Memory is a device, which is used to store information temporarity/permanently.

(a) Primary storage

- The primary storage is also called as ***"main memory"*** stores and access information very fastly. This is generally used to hold the program being currently executed in the computer, the data being received from the input unit, the intermediate and final results of the program.
- Primary storage is also known as ***system memory, internal, temporary*** and ***"RAM"***
- Installed on the main computer board (motherboard)

(b) Secondary storage

- The secondary storage is also known as ***Auxiliary Storage*** it may store several programs, documents, databases etc.
- The program that we want to run on the computer is first transferred to the primary memory before it can run. Similarly, after running the program if need to save the result, we will transfer them to the secondary storage.
- The secondary memory is slower and cheaper than the primary memory. Some of the commonly used secondary memory devices are ***Floppy diskette, Zip diskette, Hard disk and Magnetic disks*** and ***Tapes*** etc.

(c) Output unit

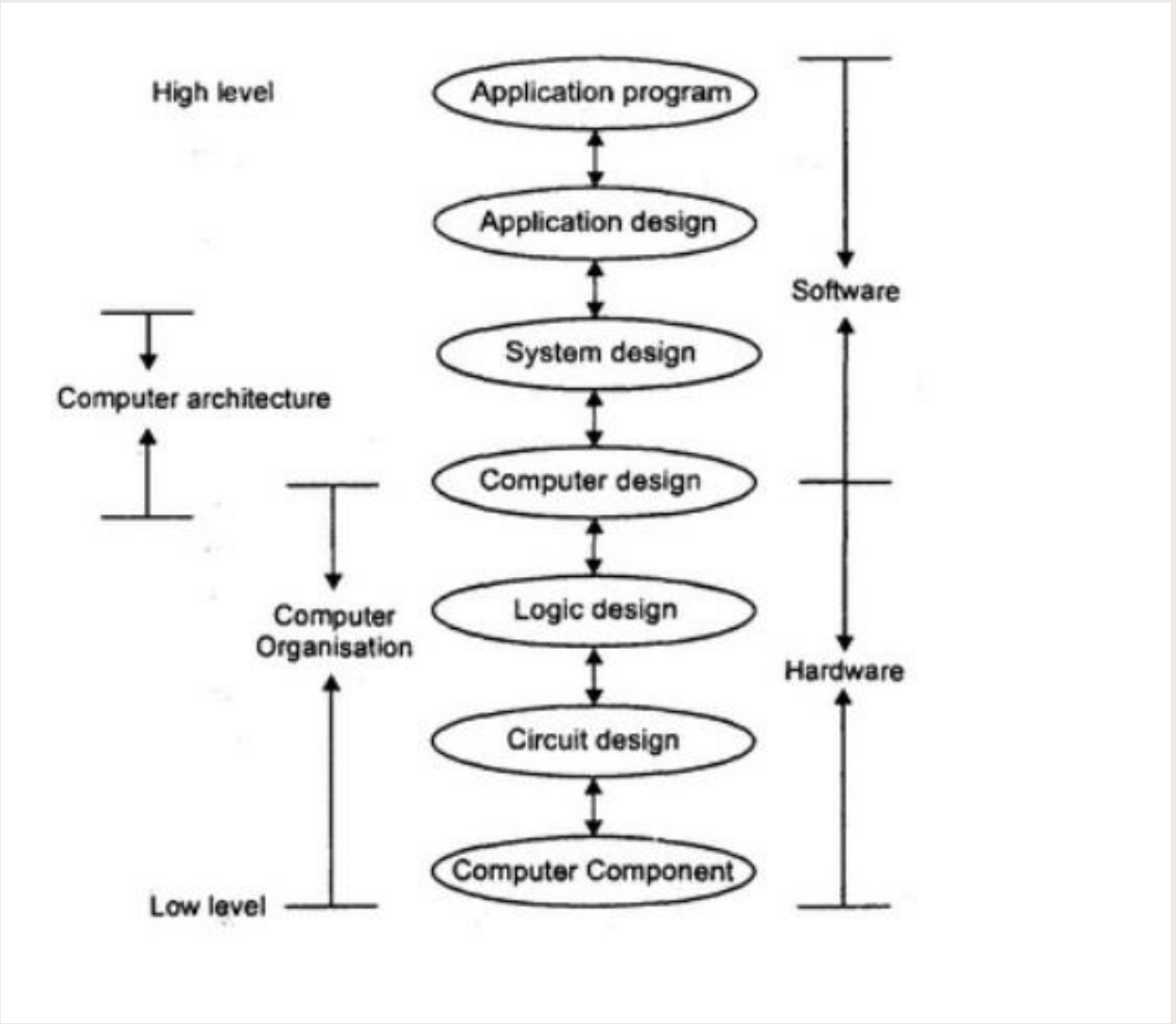
- Devices used to get the response or result of a process from the computer is called output unit is the communication between the user and the computer.
- The output unit of a computer provider the information and results of a computation to the outside world.
- Computers do not work in the decimal system, they work in the binary system. Therefore if required, the output unit also converts the binary data into a form that users can understand.
- Commonly used output devices are.
    1. Visual display unit (***VDU***) or monitor
    2. Printer
    3. Computer output microfilm
    4. Plotter.

Computer Architecture

- Computer architecture is the functional design and structure of a computer system. It focuses on how a computer's hardware components, like the CPU and memory, work together to execute instructions efficiently. It involves decisions about the instruction set, data paths, and control units to optimize performance and functionality.
- Architecture describes ***what a computer does*** and serves as the blueprint for designing computer systems. It is developed before computer organization during the system design process.

Computer Organization

- Computer organization focuses on the physical implementation of a computer system based on its architecture. It deals with how different hardware components, like the CPU, memory, and input/output devices, are connected and work together to execute tasks. While computer architecture explains **what a computer does**, computer organization describes **how it does it**.
- Computer organization ensures the architectural design is translated into a functional, physical system.


High level
Application program
Application design
System design
Computer design
Logic design
Circuit design
Computer Component
Low level
Software
Hardware
Computer architecture
Computer Organisation

Computer architecture vs computer organization

Computer Architecture	Computer Organization
Architecture describes what the computer does.	The Organization describes how it does it.
Computer Architecture deals with the functional behavior of computer systems.	Computer Organization deals with a structural relationship.
It deals with high-level design issues. (System and Computer)	It deals with low-level design issues. (Logic and circuit)
Architecture indicates its hardware.	Whereas Organization indicates its performance.
Architecture is a series of instructions, addressing modes, and registers.	The implementation of the architecture is called organization.
For designing a computer, its architecture is fixed first.	For designing a computer, an organization is decided after its architecture.
Computer Architecture comprises logical functions such as instruction sets, registers, data types, and addressing modes.	Computer Organization consists of physical units like circuit designs, peripherals, and adders.
It makes the computer's hardware visible.	It offers details on how well the computer performs.
Architecture coordinates the hardware and software of the system.	Computer Organization handles the segments of the network in a system.
The software developer is aware of it.	It escapes the software programmer's detection.
Examples- Intel and AMD created the x86 processor. Sun Microsystems and others created the SPARC processor. Apple, IBM, and Motorola created the PowerPC.	Organizational qualities include hardware elements that are invisible to the programmer, such as interfacing of computer and peripherals, memory technologies, and control signals.

Computer Architecture vs Computer Organization



Computer Architecture

What to do?

- Computer Architecture refers to how hardware components connect to form a computer system.
- It's a design blueprint & decided first.
- It acts as an interface between hardware and software.

Computer Organizations

How to do?

- Computer Organization refers to the structure and behavior of the computer system as seen by the user.
- It's decided after computer architecture.
- It t deals with the interconnection of components.

Computer Architecture vs. Computer Organization

Computer **Architecture** and **Computer Organization** are closely related concepts, but they focus on different aspects of a computer system.

1. Computer Architecture – "What does the computer do?"

Computer architecture refers to the **design and functional behavior of a computer system as visible to the programmer or user**.

- It defines **what the computer system is supposed to do**.
- It describes the **functional specifications** of the computer.
- It acts as a **design blueprint** for the computer.
- It defines how **hardware and software interact**.

Architecture is generally decided **before the detailed hardware implementation**.

Example: Deciding whether a processor supports **32-bit or 64-bit instructions** is an architectural decision.

2. Computer Organization – "How does the computer do it?"

Computer organization refers to the **way the hardware components are arranged, connected, and operated to implement the computer architecture**.

- It describes **how the computer system performs its functions**.
- It focuses on the **internal implementation of the system**.
- It deals with the **interconnection and operation of hardware components**.
- Organization explains how the components **work together to execute instructions**.
- It is concerned with the **hardware implementation of the architecture**.
- Example: Designing the **cache size, bus structure, or pipeline stages** is an organizational decision.

Example
Architecture = What?
"The processor supports an ADD instruction."
Organization = How?
"The ALU, registers, control unit, and internal buses work together to execute ADD."

Computer Architecture	Computer Organization
Describes **what the computer does**.	Describes **how the computer does it**.
Deals with **functional behavior**.	Deals with **physical implementation**.
Concerned with **high-level design**.	Concerned with **low-level design**.
Visible to the **programmer**.	Mostly **invisible to the programmer**.
Includes **instruction set, registers, data types, and addressing modes**.	Includes **ALU, control unit, memory, buses, and control signals**.
Defines the **Instruction Set Architecture (ISA)**.	Implements the **ISA through hardware**.
Determines what instructions and operations the processor can perform.	Determines how efficiently those instructions are executed.
Architecture is generally **decided first**.	Organization is designed **after the architecture**.
Focuses on **capabilities and functionality**.	Focuses on **performance, cost, and implementation**.
Example: **x86, ARM, RISC-V** instruction sets.	Example: **cache size, pipeline stages, ALU design, bus structure**.

Eight great ideas in computer architecture.

The eight great ideas in computer architecture, popularized by Patterson and Hennessy, are foundational design principles that guide how modern hardware and systems are built to maximize performance, efficiency, and reliability. [1]

Core Design Principles

- **Design for Moore's Law**: Anticipate rapid technological changes and transistor count doubling so future systems benefit.
- **Use Abstraction to Simplify Design**: Hide low-level details across different layers to manage complexity and boost productivity.
- **Make the Common Case Fast**: Focus optimization efforts on frequently used operations rather than rare events.
- **Performance via Parallelism**: Execute multiple operations simultaneously to speed up execution time.
- **Performance via Pipelining**: Split instructions into sequential stages processed like an assembly line.
- **Performance via Prediction**: Guess outcomes and start tasks early instead of waiting, provided recovery is cheap.
- **Hierarchy of Memories**: Balance speed, capacity, and cost using fast small memory on top and slow large storage below.
- **Dependability via Redundancy**: Add duplicate components or backup systems to handle and recover from failures.

8 Great Ideas in Computer Architecture

The eight great ideas in computer architecture, popularized by Patterson and Hennessy, are foundational design principles that guide how modern hardware and systems are built to maximize performance, efficiency, and reliability

Core Design Principles

1. Design for Moore's law
2. Use abstraction to simplify design
3. Make the common case fast
4. Performance via parallelism(Think about the things running in parallel)
5. Performance via pipelining(Think about the things quickly running one after other)
6. Performance via prediction(Hardware)
7. Hierarchy of memories
8. Dependability via redundancy

1. Design for Moore’s Law

Design for Moore’s Law

- **Moore’s Law** predicts that IC resources roughly **double every 18–24 months**.
- Computer architects must **design for future technology**, not just current capabilities.
- During a long design cycle, computing resources may **double or quadruple** between the start and finish of the project.
- Hence, computer design must **anticipate rapid technological change**.



💡 Everyday Analogy

Planning a city's highway system based on the population size expected in 10 years, rather than the number of people living there today.

2. Abstraction to Simplify Design

Use Abstraction to Simplify Design

- **Abstraction** hides unnecessary lower-level details and provides a **simpler view** of a system.
- It allows computer architects and programmers to **manage complex designs efficiently**.
- Different **levels of abstraction** represent a system from high-level concepts to low-level implementation.
- This improves **productivity, understanding, and design speed**.



💡 Everyday Analogy

Driving a car. You only interact with the steering wheel and pedals. You don't need to know the chemistry of the engine to travel.

3. Make the Common Case Fast

- Focus on **optimizing frequently used operations** rather than rare cases.
- Improving the common case can provide a **greater overall performance improvement**.
- Identify common operations through **experimentation, measurement, and performance analysis**.
- Since common cases are often simpler, they are usually **easier to optimize**.

💡 Everyday Analogy

Optimizing a daily commuter car for city gas mileage and parking convenience rather than its top speed capability on a race track.



4. Performance via Parallelism

Performance via Parallelism

- **Parallelism** improves performance by performing **multiple operations at the same time**.
- Computer systems use parallel processing to **complete tasks faster**.
- Examples include **multi-core CPUs, GPUs, and parallel processors**.
- Like a plane with **multiple jet engines**, more parallel resources can provide **higher performance**.

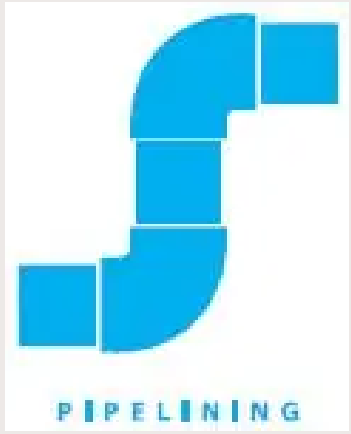


💡 Everyday Analogy

A busy grocery store opening up four checkout lanes instead of making every single shopper wait in one massive line.

5. Performance via Pipelining

- **Pipelining** is a special form of parallelism where a task is divided into **multiple stages**.
- Different stages can work on **different parts of multiple tasks simultaneously**.
- This increases **throughput** and allows more tasks to be completed in less time.



💡 Everyday Analogy

A modern factory assembly line. Workers do not wait for one full vehicle to finish before starting to construct the frame of the next one.

6. Performance via Prediction

- **Prediction** improves performance by **guessing what will happen next** and starting work in advance.
- It avoids waiting for information to be known with certainty.
- If the prediction is correct, **time and performance are improved**.
- If it is wrong, the system **recovers and performs the correct operation**.



💡 Everyday Analogy

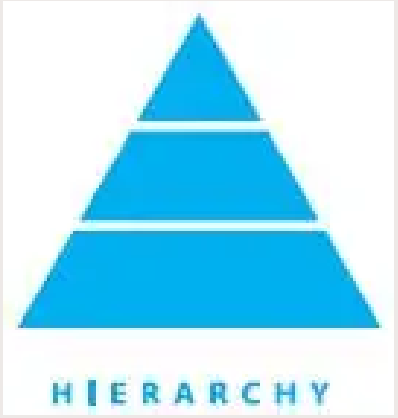
Checking a weather app and packing an umbrella. If it rains, you stay dry. If it stays sunny, you lose minimal effort carrying it.

7. Hierarchy of Memories

- Memory must be **fast, large, and inexpensive**, but achieving all three together is difficult.
- A **memory hierarchy** uses different types of memory, from fast/small to slow/large.
- **Caches** provide fast access to frequently used data and make main memory appear faster.
- The hierarchy balances **speed, capacity, and cost** for better overall performance.

💡 Everyday Analogy

Your work desk. Active papers sit directly on top (Cache). Daily folders reside in a cabinet (RAM). Old archives sit down in the basement (Hard Drive).



8. Dependability via Redundancy

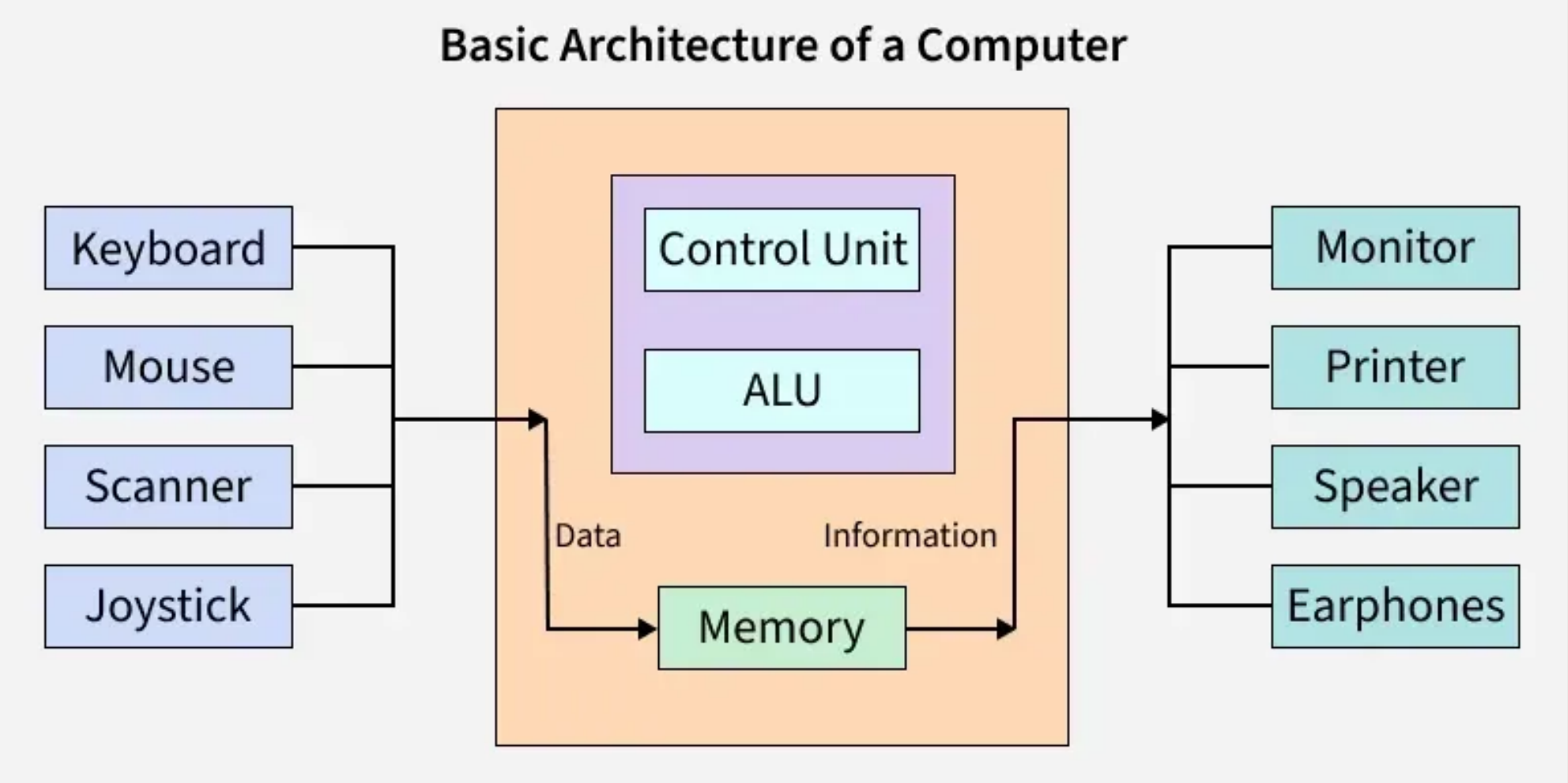
- **Redundancy** improves system dependability by providing **backup components**.
- If one component fails, another can **take over its function**.
- Redundancy also helps in **detecting and recovering from failures**.
- Example: **Dual tires on a truck** allow it to continue operating even if one tire fails



💡 Everyday Analogy

Commercial planes flying with multiple engines. If one engine fails mid-flight, the aircraft safely flies and lands using the remaining ones.

Block diagram of a computer system

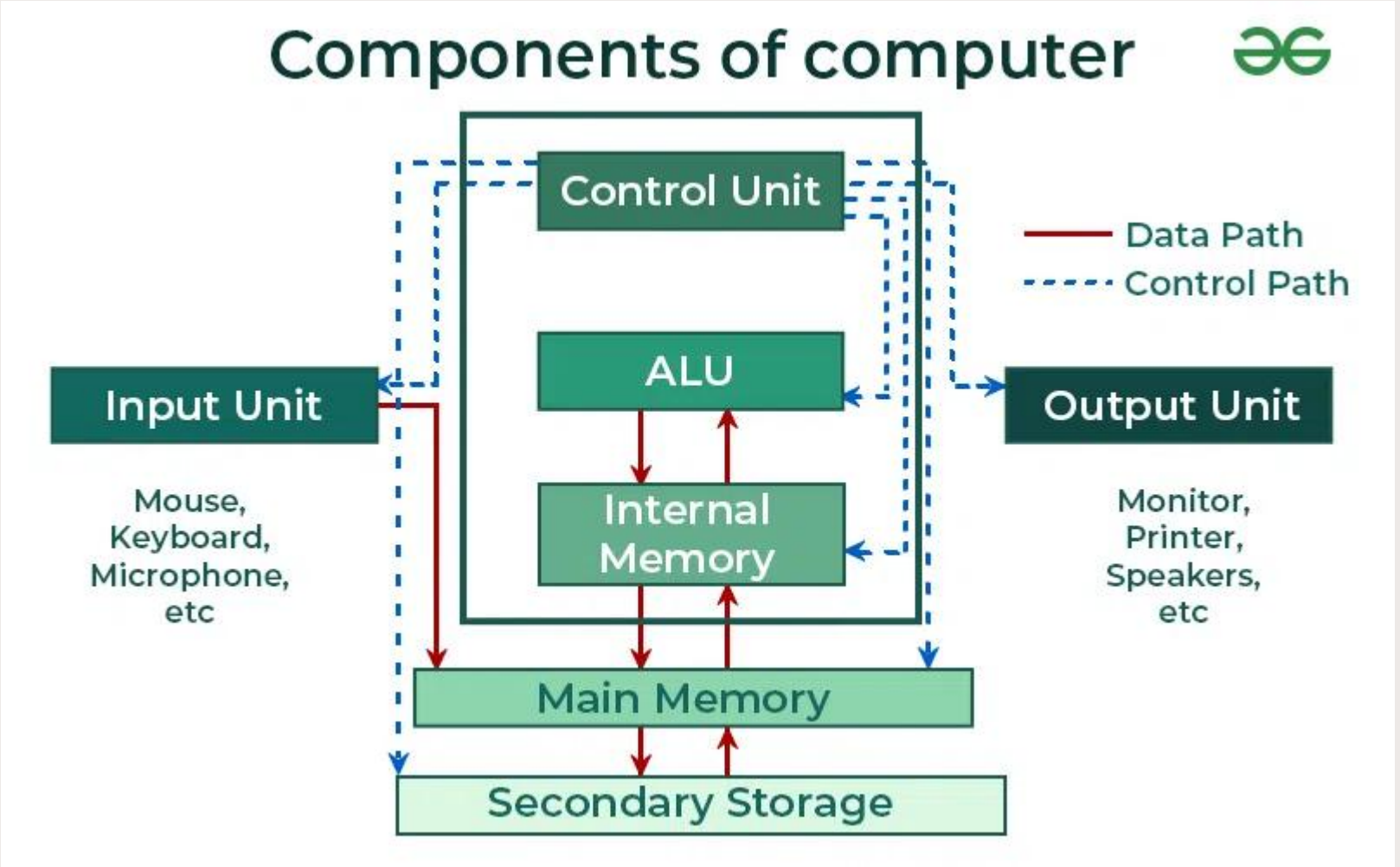


Basic Architecture of a Computer

- **Input Unit:** Accepts data and instructions from devices such as **keyboard, mouse, scanner, and joystick**.
- **Memory Unit:** Stores **data, instructions, and intermediate results** required during processing.
- **Control Unit (CU):** Controls and coordinates the activities of all computer components.
- **Arithmetic Logic Unit (ALU):** Performs **arithmetic operations** and **logical/comparison operations**.
- **Output Unit:** Presents processed information through devices such as **monitor, printer, speaker, and earphones**.
- **Data Flow:** Input data → **Memory/Processing (CU + ALU)** → Output information.

- **Overall Function: The computer accepts input, processes and stores data, and produces meaningful output.**

Functional units of a computer system



Functional Components of a Digital Computer

- **Input Unit**
  - Accepts **data and instructions** from the user.
  - Sends input to the computer for processing.
- **Central Processing Unit (CPU)**
  - Acts as the **brain of the computer**.
  - Performs processing and controls all operations.
  - Main components:
    - **Control Unit (CU)** – controls and coordinates operations.
    - **Arithmetic Logic Unit (ALU)** – performs arithmetic and logical operations.
    - **Registers** – provide fast temporary storage during processing.
- **Memory Unit**
  - Stores **data, instructions, and results**.
  - **RAM** – temporary/volatile storage.
  - **HDD/SSD** – permanent/non-volatile storage.
- **Output Unit**
  - Presents the **processed results** to the user.
  - Examples: **Monitor, Printer, Speakers**.
  - **Bus System**
  - Provides communication between computer components.
  - Transfers **data, addresses, and control signals**.

Central Processing Unit (CPU)

1. Definition

- **CPU** stands for **Central Processing Unit**.
- It is the **main processing component** of a computer.
- It is commonly known as the **"brain of the computer."**
- The CPU executes instructions provided by computer programs.
- It performs calculations, makes logical decisions, and controls other components.

2. Main Functions of CPU

- **Fetches** instructions from memory.
- **Decodes** the instructions to understand what operation is required.
- **Executes** the instructions.
- **Stores** intermediate and final results.
- Controls the operation of **Input, Output, and Memory units**.
- Manages the allocation of **computing resources**.
- Performs multiple operations required by the **Operating System and applications**.

3. Major Components of CPU

A. Control Unit (CU)

- Controls and coordinates all activities of the computer.
- Fetches instructions from memory.
- Decodes the instructions.
- Generates **control signals** for different components.
- Determines the sequence in which operations are performed.

B. Arithmetic Logic Unit (ALU)

- Performs **arithmetic operations**:
  - Addition
  - Subtraction
  - Multiplication
  - Division
- Performs **logical operations**:
  - AND
  - OR
  - NOT
  - XOR
- Performs **comparison operations**:
  - Greater than
  - Less than
  - Equal to
- Sends the calculated results to registers or memory.

C. Registers

- Small and **very fast storage locations** inside the CPU.
- Temporarily holds data, addresses, and instructions for immediate use
- Provide data to the CPU at very high speed.

- Examples:
  - **Program Counter (PC)** – stores the address of the next instruction.
  - **Instruction Register (IR)** – stores the current instruction.
  - **Accumulator (ACC)** – Holds the results of math and logic tasks..
  - **Memory Address Register (MAR)** – stores the address of a memory location.
  - **Memory Data Register (MDR)** – stores data being transferred to/from memory.

4. CPU Instruction Cycle

Fetch → Decode → Execute → Store

- **Fetch:** CPU retrieves an instruction from memory.
- **Decode:** Control Unit interprets the instruction.
- **Execute:** ALU or another CPU component performs the required operation.
- **Store:** Result is stored in a register or memory.

Example

When a user calculates **25 + 15**:

1. Input values **25 and 15** are provided.
2. CPU fetches the required instruction.
3. **Control Unit** decodes the instruction.
4. **ALU** performs the addition.
5. Result **40** is stored temporarily in a register.
6. The result is sent to the **Output Unit** and displayed.

Other Components

Input Unit → Memory → CPU → Output Unit

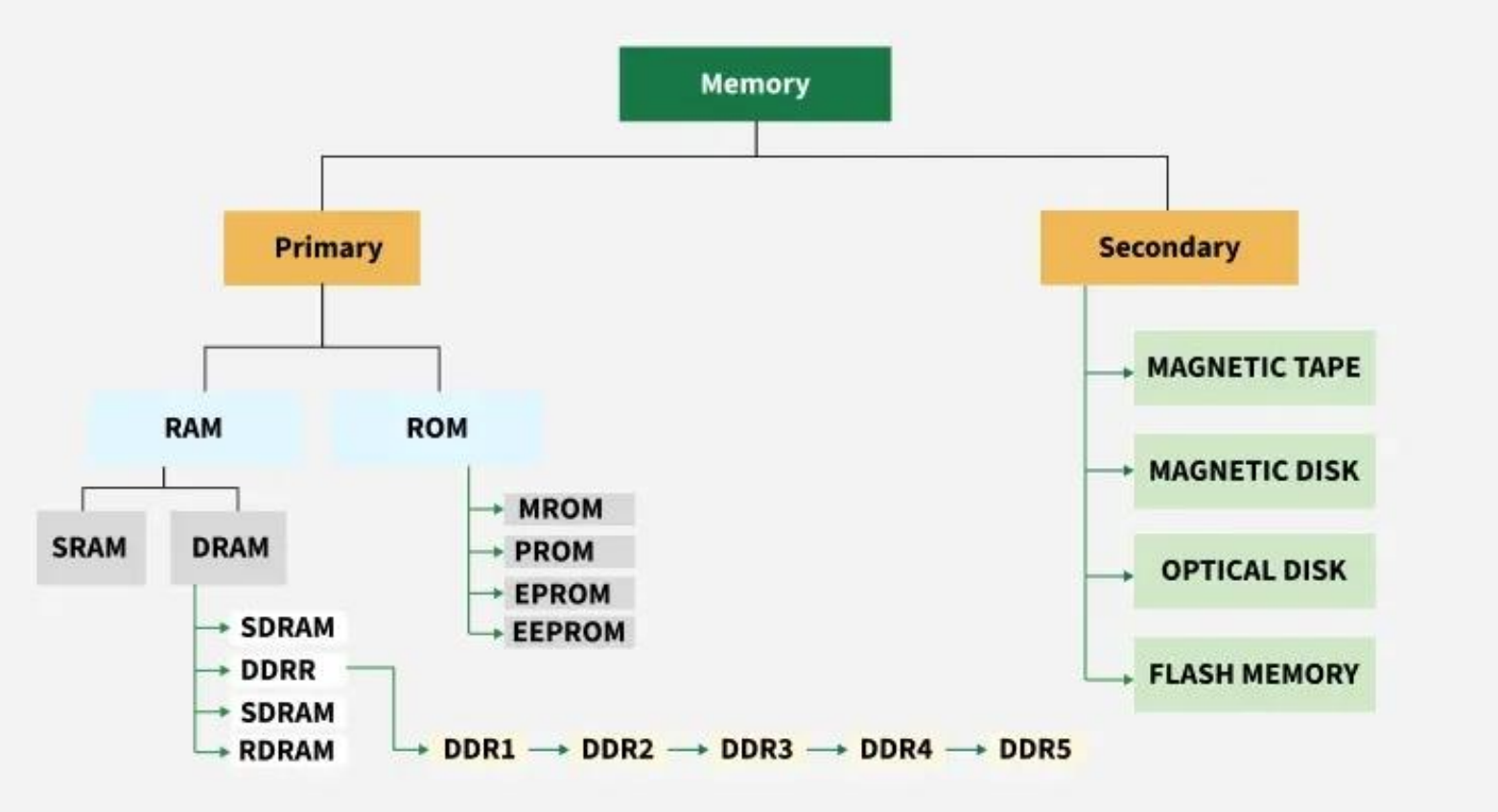
- **Input Unit:** Provides data and instructions.
- **Memory:** Stores programs, data, and results.
- **CPU:** Processes data and executes instructions.
- **Output Unit:** Displays or communicates the processed results.
- **Bus System:** Transfers data, addresses, and control signals between components.

6. Important Characteristics

- **Processing Speed:** Determines how quickly instructions can be executed.
- **Clock Speed:** Measured in **Hz, MHz, or GHz**; indicates the clock cycles per second.
- **Number of Cores:** Multiple cores allow several tasks to be processed simultaneously.
- **Cache Memory:** High-speed memory that stores frequently used data and instructions.
- **Word Length:** Number of bits the CPU can process at a time, such as **32-bit or 64-bit**.
- **Instruction Set:** Defines the operations that the CPU can understand and execute.

Memory

- Memory is the electronic storage space where a computer keeps the instructions and data it needs to access quickly.
- It's the place where information is stored for immediate use.
- Memory is an important component of a computer, as without it, the system wouldn't operate correctly.
- The computer's operating system (OS), hardware, and software all rely on memory to function properly.



Memory

- **Memory** is used to store **data, instructions, and results** in a computer.
- It is broadly classified into:
    1. **Primary Memory**
    2. **Secondary Memory**

1. Primary Memory

- Also called **main memory**.
- Directly accessed by the **CPU**.
- Provides **fast access** to data and instructions.
- Mainly includes:
    - **RAM**
    - **ROM**

RAM – Random Access Memory

- RAM is a **read/write memory**.
- Stores programs and data currently being used by the CPU.
- It is generally **volatile**, meaning data is lost when power is turned off.

Two major types:

a) **SRAM**
b) **DRAM**

a) SRAM – Static RAM

- Stores data using **flip-flop circuits**.
- Faster than DRAM.
- Does not require periodic refreshing.
- More expensive and commonly used in **CPU cache memory**.

b) DRAM – Dynamic RAM

- Stores data using **capacitors and transistors**.
- Requires periodic **refreshing**.
- Slower than SRAM but **cheaper and has higher storage capacity**.
- Commonly used as the computer's **main memory**.

Types of DRAM

- **SDRAM – Synchronous DRAM**
  - Operates in synchronization with the CPU/system clock.
- **DDR – Double Data Rate SDRAM**
  - Transfers data on both the rising and falling edges of the clock.
- Generations of DDR include:
  - **DDR1**
  - **DDR2**
  - **DDR3**
  - **DDR4**
  - **DDR5**
- Each newer generation generally provides **higher speed, greater bandwidth, and improved power efficiency**.

- **PPT Note:** The image appears to contain a few labeling/typing inconsistencies in the DRAM branch. The standard sequence is **SDRAM → DDR → DDR1 → DDR2 → DDR3 → DDR4 → DDR5**.

ROM – Read Only Memory

- ROM stores **permanent or semi-permanent instructions**.
- It is **non-volatile**, so data remains even when power is switched off.
- Traditionally used to store **firmware and boot instructions**.

Types of ROM

a) **MROM (Mask ROM)**
   - Programmed during manufacturing.

b) **PROM (Programmable ROM)**
   - Can be programmed **once** by the user.

c) **EPROM (Erasable Programmable ROM)**
   - Can be erased using **ultraviolet (UV) light** and reprogrammed.

d) **EEPROM (Electrically Erasable Programmable ROM)**
   - Can be erased and reprogrammed **electrically**.

Secondary Memory

- Used for **long-term/permanent storage** of data.
- It is **non-volatile**.
- Provides **larger storage capacity** than primary memory.
- Usually slower than primary memory.
- Examples shown in the image:
    a) **Magnetic Tape**
    b) **Magnetic Disk**
    c) **Optical Disk**
    d) **Flash Memory**

a) Magnetic Tape

- Stores data using **magnetic material** on a tape.
- Mainly used for **backup and archival storage**.
- Provides high capacity but **slower access** because it is sequential.

b) Magnetic Disk

- Uses magnetically coated rotating disks to store data.
- **Hard Disk Drive (HDD)** is a common example.
- Provides large storage capacity at relatively low cost.

c) Optical Disk

- Stores data using **laser technology**.
- Examples:
  - **CD**
  - **DVD**
  - **Blu-ray Disc**
- Used for distributing and storing digital media.

d) Flash Memory

- **Non-volatile semiconductor memory**.
- Faster and more portable than traditional magnetic storage.
- Examples:
  1. **SSD**
  2. **USB Flash Drive**
  3. **Memory Card**

Memory
↓
Primary Memory → RAM + ROM
Secondary Memory → Magnetic Tape + Magnetic Disk + Optical Disk + Flash Memory

Primary Memory	Secondary Memory
Directly accessed by CPU	Not directly accessed in the same way as main memory
Faster	Comparatively slower
Smaller capacity	Larger capacity
RAM is volatile; ROM is non-volatile	Generally non-volatile
Example: RAM, ROM	Example: HDD, SSD, USB, DVD

Programs

Definition

A program is a set or sequence of instructions given to a computer to perform a specific task or solve a particular problem.

Simple Computer Example:

Suppose we want a computer to **add two numbers**.

We need to give instructions such as:

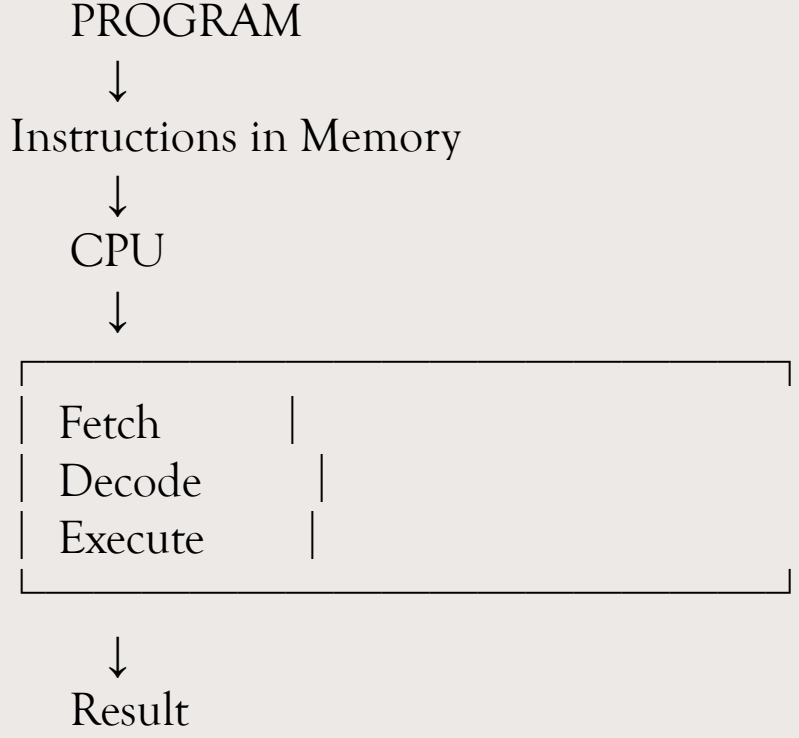
1. Take the first number
2. Take the second number
3. Add the two numbers
4. Store the result
5. Display the result

These instructions together form a **program**

But how does the computer actually execute this program?"

The program is stored in **memory** as instructions.

The **CPU** reads these instructions one by one and executes them.



For example, a simple program may perform:

```
LOAD  A
ADD   B
STORE C
HALT
```

This means:

1. Load the value of **A** into a CPU register.
2. Add the value of **B**.
3. Store the result in **C**.
4. Stop execution.

What is an Instruction?

Each individual step in a program is called an instruction.
For example:
ADD A, B

This is one **instruction**.

An instruction generally tells the CPU:
What to do + With what data + Where the data/result is located

This gives us three important concepts:
1. Operation
2. Operand
3. Address / Location

What is Operation?

Simple definition
Operation means the action that the CPU has to perform.
In other words:
Operation = What should the CPU do?
Examples:

Operation	Meaning
ADD	Add two values
SUB	Subtract
MUL	Multiply
DIV	Divide
LOAD	Bring data into CPU/register
STORE	Save data to memory

Simple example
ADD A, B
Here:
ADD is the **operation**.
It tells the CPU:
“Perform addition.”

What is Operand?

Simple definition
Operand is the data on which the operation is performed.

In simple words:
Operand = On what should the CPU perform the operation?
Example:
ADD A, B
Here:

- **ADD** → Operation
- **A and B** → Operands

Suppose:
A = 10
B = 20
Then:
ADD A, B
means:
10 + 20 = 30
So **10 and 20 are the actual data**, while **A and B are their names/locations**.

What is Address / Location?

Address is the location where data or an instruction is stored in memory.

Real-life example

Imagine a college classroom.

If I say:

"Go to Room No. 205 and bring the attendance register."

Room No. 205 tells you **where to find the register**.

Similarly, in computer memory, every location has an **address**.

For example:

Memory

Address	Data
1000	10
1001	20
1002	30
1003	40

If the CPU wants the value **20**, it goes to memory address **1001**.

So:

Address = Where is the data stored?

Example: Adding Two Numbers

Suppose:

A = 10

B = 20

We want:

C = A + B

The CPU can perform operations conceptually like:

LOAD A

LOAD B

ADD

STORE C

Instruction	What CPU does
LOAD A	Gets value of A
LOAD B	Gets value of B
ADD	Adds the values
STORE C	Stores result in C

Finally:

C = 30

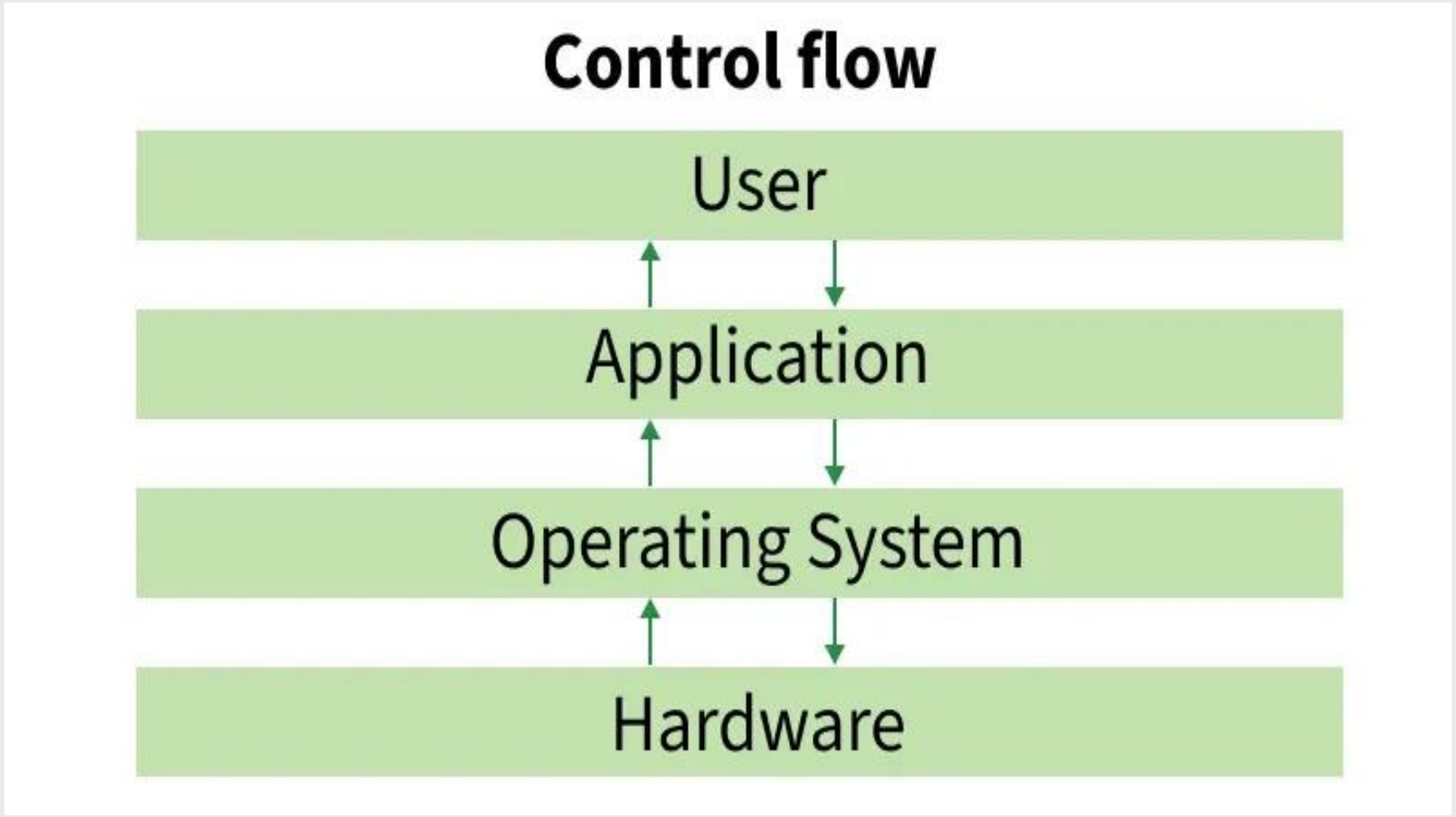
Introduction to Operating System

An operating system acts as an interface between the computer hardware and the user. It prevents the user from directly accessing the hardware and manages all system resources.

- It provide an environment in which a user can execute programs conveniently and efficiently.
- Operating system (OS) is a program that runs at all times on a computer.
- It assigns resources such as memory, processors, and input/output devices to processes that need them.

Operating System
OS = System Software
Acts as an **interface between user and hardware**
Manages computer resources
Provides an environment to **run applications efficiently**
Example: Windows, Linux, macOS, Unix


Windows
mac OS
macOS
Linux
ChromeOS
Examples of OS
Android
IOS
UNIX
An Open Group Standard
UNIX
BSD
BSD


Control flow
User
Application
Operating System
Hardware

Architecture of OS/ Operating System and User Interaction

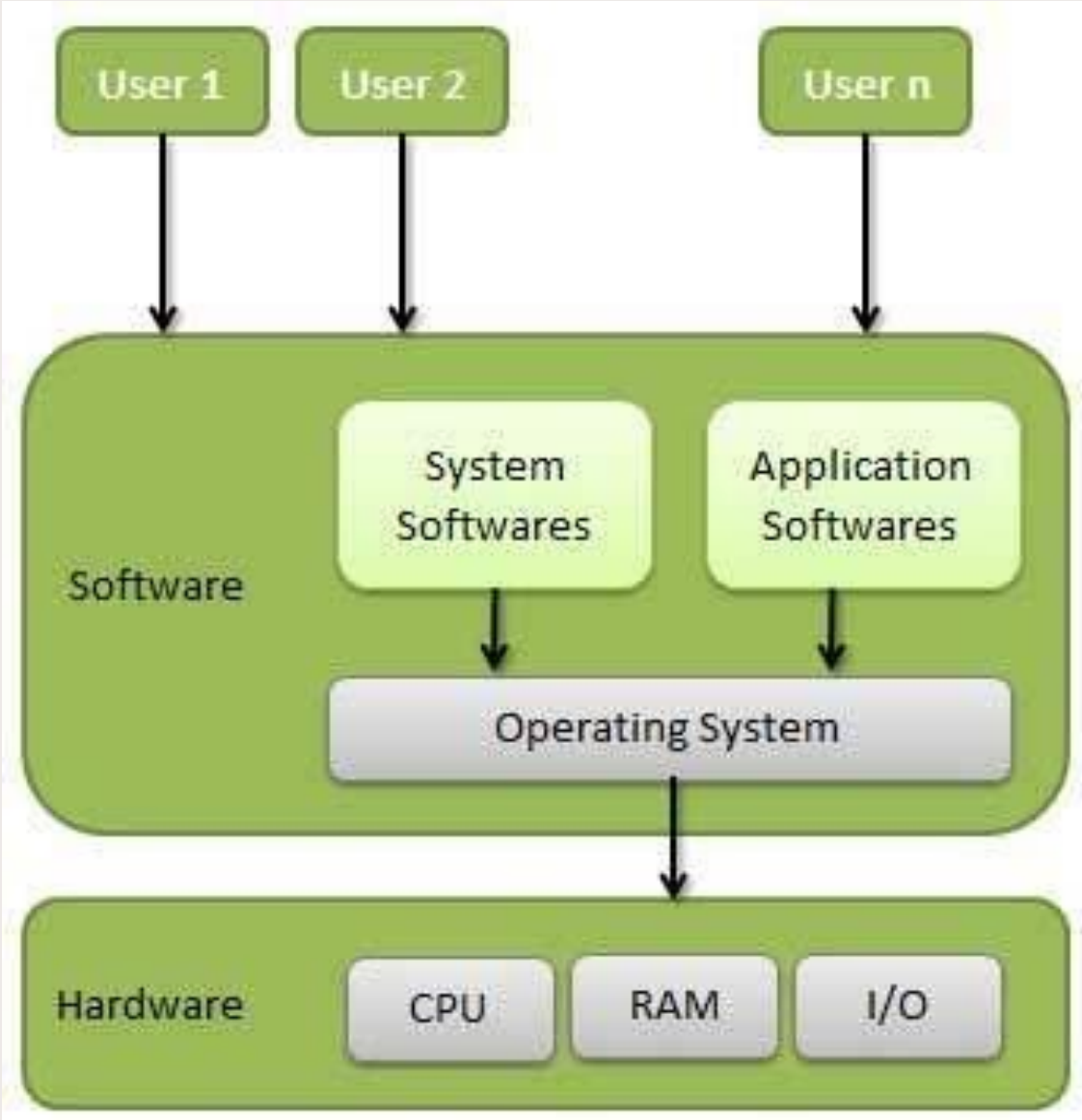
We can draw a generic architecture diagram of an Operating System which is as follows –

This diagram shows **how users interact with computer hardware through different layers of software**. It is a very good example of **abstraction** in computer architecture.

The computer system is divided mainly into **two parts**:

1. **Software**
2. **Hardware**

The flow is:
Users → Application/System Software → Operating System → Hardware



1. 👤 USER – Who uses the computer?

At the top, we have:
User 1, User 2, … User n
A user is a person who interacts with a computer system.
For example:

- A student writing an assignment
- A teacher preparing a PPT
- A programmer writing Python code
- A person watching a YouTube video
- An employee preparing an Excel sheet

What does the user do?
The user gives commands such as:

- Open a file
- Play a video
- Print a document
- Calculate something
- Save a file
- Browse a website

The important point is:
A user normally does not know how the hardware performs these operations internally.
For example, when you click **Print**, you don't tell the CPU how to communicate with the printer. Software handles it for you.

2. 📱 APPLICATION SOFTWARE

Application software is the software that the **user directly uses to perform a particular task**.

Examples
the operating system take care of those details.

Application	**Purpose**
MS Word	Create documents
PowerPoint	Create presentations
Chrome	Browse websites
VLC	Play videos
Calculator	Perform calculations
Photoshop	Edit images
Python IDE	Write programs

Example
Suppose a student wants to write an assignment.
The student opens **MS Word**.
Student
↓
MS Word

The student types:
Computer Architecture is the study of...

The user doesn't need to know how the characters are stored in RAM or how the CPU processes them.

MS Word and the operating system take care of those details.

3. ⚙️ SYSTEM SOFTWARE

System software works more closely with the computer's hardware.
Its job is to **support the operation and management of the computer system**.
Examples include:

- Operating systems
- Device drivers
- Language translators
- System utilities

Device Driver – Simple Example

Suppose you connect a printer to your computer.
The computer needs software that knows how to communicate with that particular printer.
This software is called a **device driver**.

Application
↓
Operating System
↓
Printer Driver
↓
Printer

The driver acts like a **translator** between the operating system and the hardware device.

Simple analogy

Imagine an Indian person speaking Marathi and a person speaking Japanese.
A translator helps them communicate.

Similarly:
Device driver = Translator between OS and hardware device

4. 🪟 OPERATING SYSTEM

This is the **most important layer in this diagram**.

The operating system acts as an **interface between application software and hardware**.

Examples:

- Windows
- Linux
- macOS
- Android
- iOS

What does the OS do?

The operating system manages computer resources such as:

CPU Management

It decides:

Which program should get CPU time?

For example, you may have:

- Chrome running
- MS Word running
- Music playing

The OS manages CPU resources among them.

Memory Management

Suppose you open:

- Chrome
- Word
- PowerPoint

All these programs need RAM.

The OS decides:

How much memory should each program use?

File Management

When you save:

Assignment.docx

the OS manages where and how that file is stored.

I/O Management

The OS manages communication with:

- Keyboard
- Mouse
- Printer
- Monitor
- Speakers
- USB devices

5. 🖥️ HARDWARE

Hardware means the **physical components of the computer**.
You can touch hardware.
Examples:

- CPU
- RAM
- Keyboard
- Mouse
- Monitor
- SSD
- Printer
- Speakers

In diagram, three major hardware components are shown:

CPU + RAM + I/O

🧠 CPU

CPU stands for **Central Processing Unit**.
It is responsible for:

- Executing instructions
- Performing calculations
- Making decisions
- Controlling operations

CPU is the main processing unit of a computer.

Example
Suppose we have:
5 + 3
The CPU performs the required arithmetic operation and produces:
8

But remember: the CPU doesn't just perform arithmetic. It executes **instructions** given by programs.

💾 RAM

- RAM stands for **Random Access Memory**.
- It is the computer's **temporary working memory**.
- When you open a program, the program and the data it is currently using are loaded into RAM.

Example
Suppose you open Chrome.

Chrome stored on SSD
↓
Loaded into RAM
↓
CPU
↓
Executes program

RAM is much faster to access than secondary storage, but it is **volatile**.

That means:
When power is switched off, the contents of RAM are normally lost.

⌨️🖨️ I/O

I/O means **Input/Output**.
It allows the computer to communicate with the outside world.

Input devices

These provide data **to** the computer.
Examples:

- Keyboard
- Mouse
- Microphone
- Scanner
- Camera

Output devices

These receive information **from** the computer.
Examples:

- Monitor
- Printer
- Speaker
- Projector

So:
Input
Keyboard → Computer

Output
Computer → Monitor

Let's Take One Complete Example

Example: Student types "Hello" in MS Word

Step 1 – User
The student presses the **H** key.
USER
↓
Presses H

Step 2 – Application Software
MS Word is the application currently being used.
USER
↓
MS Word
MS Word needs the operating system to receive the keyboard input.

Step 3 – Operating System
The OS receives the keyboard input and manages the communication between the application and the hardware.
MS Word
↓
Operating System

Step 4 – Hardware

The keyboard is an I/O device.

The input eventually reaches the system, and the CPU processes the required instructions.

The character is then displayed on the monitor.

Keyboard

↓

Operating System

↓

MS Word

↓

RAM / CPU

↓

Monitor

The student simply sees:

H

They don't need to understand all the internal operations.

🖨️ Another Example: Printing a Document

Suppose you click:

File → Print

What happens?

User

↓

MS Word

↓

Operating System

↓

Printer Driver

↓

Printer

The user doesn't directly control the printer's electronics.

The software layers handle the communication.

This is exactly why **abstraction is useful**.

🔄 **Let's Understand the Complete Flow**

Suppose a student wants to **play a video**.

Step 1 – User

The student clicks on a video.

⬇️

Step 2 – Application Software

VLC or another video player receives the request.

⬇️

Step 3 – Operating System

The application asks the OS to access the video file.

⬇️

Step 4 – Hardware

The OS communicates with hardware such as:

- CPU → processes instructions
- RAM → stores data temporarily
- Storage/I/O → provides the video data and sends output to the display/speaker

🧠 Very Simple Real-Life Analogy

Think about **ordering food in a restaurant**:

You (User)

↓

Waiter (Application)

↓

Kitchen Manager (Operating System)

↓

Cooks & Equipment (Hardware)

You simply tell the waiter:

"I want a pizza."

You don't go into the kitchen and operate the oven yourself!

Similarly:

User → Application → OS → Hardware

The **Operating System acts as a bridge between software and hardware.**

Functions of an Operating System

Following are some of important functions of an operating System.

1. Memory Management
2. Processor Management
3. Device Management
4. File Management
5. Network Management
6. Security
7. Control over system performance
8. Job accounting
9. Error detecting aids
10. Coordination between other software and users

Memory Management : Memory management refers to management of Primary Memory or Main Memory.

What does it mean?

The OS manages the computer's **main memory (RAM)**.

When several programs are running, each program needs some RAM. The OS decides:

- Which program gets memory
- How much memory it gets
- When memory should be released
- Which program is currently using which memory

Simple example

Suppose you open:

- Chrome
- MS Word
- PowerPoint
- VLC

All of them need RAM.



The OS manages the allocation of RAM among these programs.

Processor Management

In multiprogramming environment, the OS decides which process gets the processor when and for how much time. This function is called **process scheduling**.

The CPU can execute instructions, but many programs may need the CPU at the same time.
The OS decides:

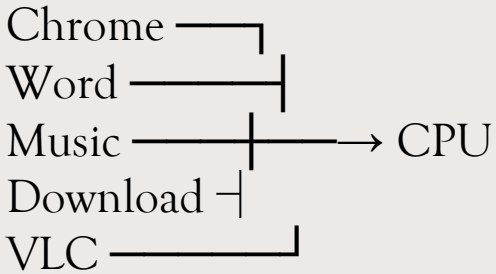
Which program gets the CPU and for how long?

For example, you may be:

- Listening to music
- Downloading a file
- Typing in Word
- Browsing Chrome

It looks as if everything is happening simultaneously.

The OS manages CPU time between these processes.



Device Management

An Operating System manages device communication via their respective drivers.

A computer has many hardware devices:

- Keyboard
- Mouse
- Printer
- Scanner
- Monitor
- Speaker
- USB drive

The OS manages communication between software and these devices.

Example: Printing

You click:

Print

The application sends a request to the OS.

MS Word

↓

Operating System

↓

Printer Driver

↓

Printer

The OS coordinates the printing operation.

File Management

The OS manages files and folders stored on storage devices such as:

- SSD
- HDD
- USB drive

It allows users to:

- Create files
- Delete files
- Rename files
- Copy files
- Move files
- Open files
- Organize files into folders

Example

You create:

AI_Assignment.docx

and save it inside:

Documents → AI → Assignments

The OS manages the storage and organization of this file.

Network Management

The OS manages communication between the computer and a network.

It helps manage:

- Wi-Fi
- Ethernet
- Internet connection
- Network settings
- Data communication

Security

The OS protects the computer and its data from unauthorized access.

It manages:

- User accounts
- Passwords
- Permissions
- Access to files
- Access to system resources

Example

Suppose you have a laptop with two users:

User A → Can access A's files

User B → Can access B's files

The OS controls who can access which files and resources.

Control Over System Performance

Recording delays between request for a service and response from the system.

What does it mean?

The OS keeps track of **how quickly the computer responds to a request**.

Whenever a user or program asks the computer to do something, there is some time between:

Request → Response

This time is called **response time** or **delay**.

Simple example
You click on a file to open it.
You click "Open"
↓
Request
↓
Computer
↓
File appears
↓
Response
Suppose the file opens after **2 seconds**.
The OS/system can monitor such delays to understand system performance.

- **Job accounting** – Keeping track of time and resources used by various jobs and users.

  What does it mean?
  The OS keeps a record of **who used the computer, what they used, and how much resources they consumed**.

Error Detecting Aids
Given statement:
Production of dumps, traces, error messages, and other debugging and error detecting aids.
This sounds difficult, so break it into two words:
Error detection + debugging
What does it mean?
When something goes wrong in a computer program or system, the OS provides information that helps us **find out what went wrong**.
For example:
Program running
↓
Error
↓
Operating System
↓
Error message / diagnostic information

Coordination Between Other Software and Users

What does it mean?
The OS coordinates **different software tools and users** so that they can use the computer resources properly.
For example, programmers may need:

- Compiler
- Interpreter
- Assembler
- Editor
- Debugger
- Other development tools

The OS helps these programs access the required CPU, memory, files and I/O devices.

- An operating system is always running in the background to manage and coordinate the use of hardware resources among system and application programs for multiple users.
- The OS itself is not only a user interface. Instead, it provides an interface for user interaction, typically through:

    Command-Line Interface (CLI), (e.g., Bash, PowerShell)

    Graphical User Interface (GUI), (e.g., Windows desktop, macOS Finder)

Goals of Operating System – Primary

- **User Convenience** – Easy and user-friendly interaction
- **Program Execution** – Provides environment to run programs
- **Resource Management** – Manages CPU, memory, storage and I/O
- **Security** – Protects data and system from unauthorized access

Goals of Operating System – Secondary

- **Efficient Resource Utilization**
  - Maximizes CPU, memory and I/O utilization
- **Reliability**
  - Handles errors gracefully
  - Provides stable and continuous operation
  - Easy to maintain and debug

Components of OS

1. Shell

- User interaction layer
- Accepts commands/input
- Displays output

2. Kernel

- Core of the OS
- Interface between **OS and hardware**
- Manages system resources

Simple flow:

User → Shell → Kernel → Hardware

Hardware Interaction

Hardware interaction means the way **programs and operating systems communicate with and control computer hardware** such as the CPU, memory, keyboard, disk, printer, and other devices.
A simple way to understand it is:
User → Application Program → Operating System → Hardware

1. Role of Programs

A **program** is a set of instructions that tells the computer what task to perform.
For example, consider a simple program:
Add two numbers
A = 10
B = 20
C = A + B
Display C
The program tells the computer:

- What operation to perform → **Addition**
- What data to use → **10 and 20**
- Where to store the result → **C**
- What output to produce → **30**

However, the program generally does **not directly control the hardware**. It requests services from the operating system.
Example:
When a program wants to save a file, it asks the operating system to write the data to the disk.

2. Role of Compilers

A **compiler** converts a program written in a **high-level language** such as C, C++, or Java into a form that the computer can execute.

For example:

int c = a + b;

The CPU cannot directly understand this high-level statement.

The compiler translates it into **machine instructions** such as:

LOAD

ADD

STORE

These instructions are represented internally as **binary (0s and 1s)**.

Basic flow

High-Level Program

↓

Compiler

↓

Machine Code / Instructions

↓

CPU

↓

Hardware

Important point:

The compiler acts as a **translator between the programmer and the machine**.

Programs tell the computer what to do, compilers translate programs into machine instructions, and the operating system manages communication between those programs and the hardware.

If you open **MS Word** and save a document:

1. **MS Word (program)** creates the document.
2. It asks the **OS** to save the file.
3. The **OS** manages the file operation.
4. The **device driver** communicates with the storage device.
5. The **storage hardware** stores the data.

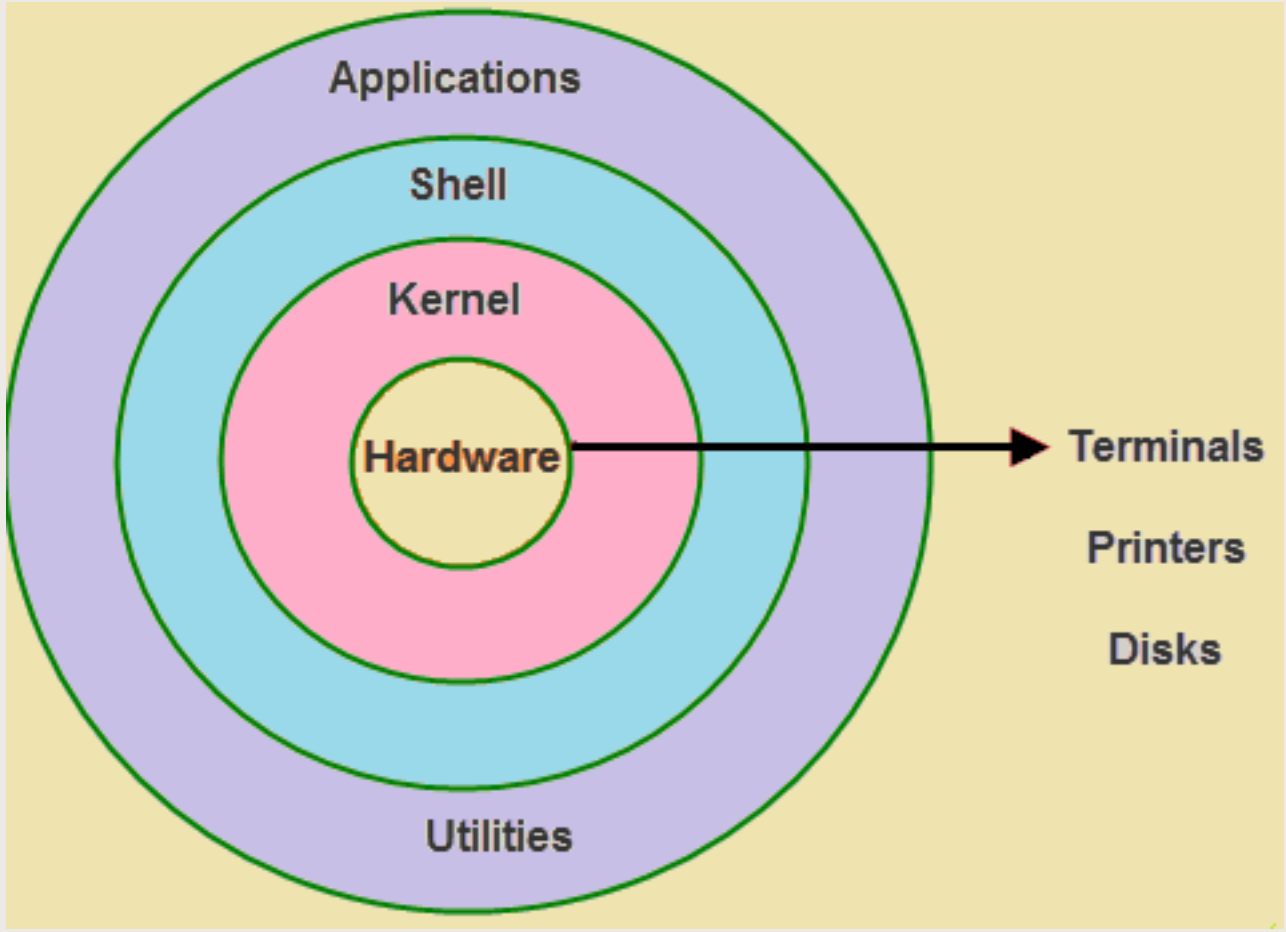
So, **the OS is the bridge between software and hardware.**

Hardware Interaction

In computing, the operating system (OS) acts as a bridge between computer hardware and the user.

Two important components of an operating system are the shell and the kernel. Both work together to ensure smooth interaction between users, applications and hardware.

- The shell provides a user interface for interacting with the operating system
- The kernel manages hardware and system resources
- Both components work together to execute user commands
- The shell operates at a higher level than the kernel
- The kernel interacts directly with computer hardware



Shell in Operating System

- **Shell – User Interface of OS**
- **Shell** acts as a **bridge between the user and the Kernel**.
- It accepts user commands and translates them into actions performed by the OS.
- A shell is a special user program that provides an interface for users to use operating system services. Shell accepts human-readable commands from a user and converts them into something which the kernel can understand.
- The shell gets started when the user logs in or start the terminal.

- The shell provides a convenient way for users to communicate with the operating system.

Types of Shells

Shells can broadly be categorized into two types:

1. Command Line Shell (CLI)

Command Line shell provides a command-line interface where users type commands to perform specific tasks. It is lightweight, faster and preferred by developers and system administrators.

2. Graphical Shell (GUI Shell)

Graphical shell provides a graphical interface to interact with the operating system using windows, icons and menus instead of typing commands.

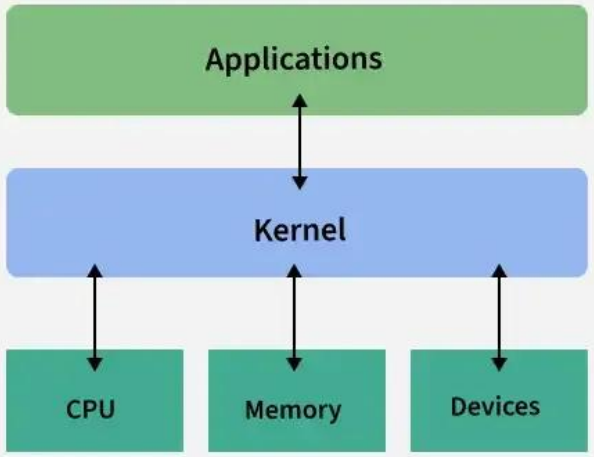
Examples:

GNOME Shell in Linux. ,Windows Explorer in Microsoft Windows.

Kernel in Operating System

The kernel is the core component of the operating system that acts as a bridge between the **hardware** and **software**, managing system resources, **controlling input/output operations**, and providing essential services to applications.

- **Core component** of the Operating System.
- Acts as a **bridge between applications and hardware**.
- Manages **CPU, memory, and I/O devices**.
- Handles **program execution and file access**.
- Controls communication with devices like **keyboard, printer, and disk**.
- Ensures **efficient and secure use of system resources**.



- The kernel is responsible for tasks like scheduling processes, managing memory, and enforcing security policies.

- It is the first program that runs when a computer starts up and remains in memory throughout the system's operations.
- Different operating systems have different kernel designs, such as the Linux kernel, Windows kernel and macOS kernel.
- The kernel plays a crucial role in maintaining stability, security, and overall functionality of the operating system.

Interaction of a Program with Hardware

How does a program reach the hardware?

1.Write Program

1. Programmer writes code using **C, C++, Java, Python**, etc.
2. Code is written in a **text editor/IDE**.

2.Compilation

1. The **compiler** translates high-level code into **assembly language** or intermediate form.

3.Assembly

1. The **assembler** converts assembly instructions into **machine code**.
2. Machine code consists of **0s and 1s (binary)**.

4.Machine Code

1. Binary instructions are understood directly by the **CPU**.
2. They control electronic circuits and transistors inside the processor.

5.Hardware Execution

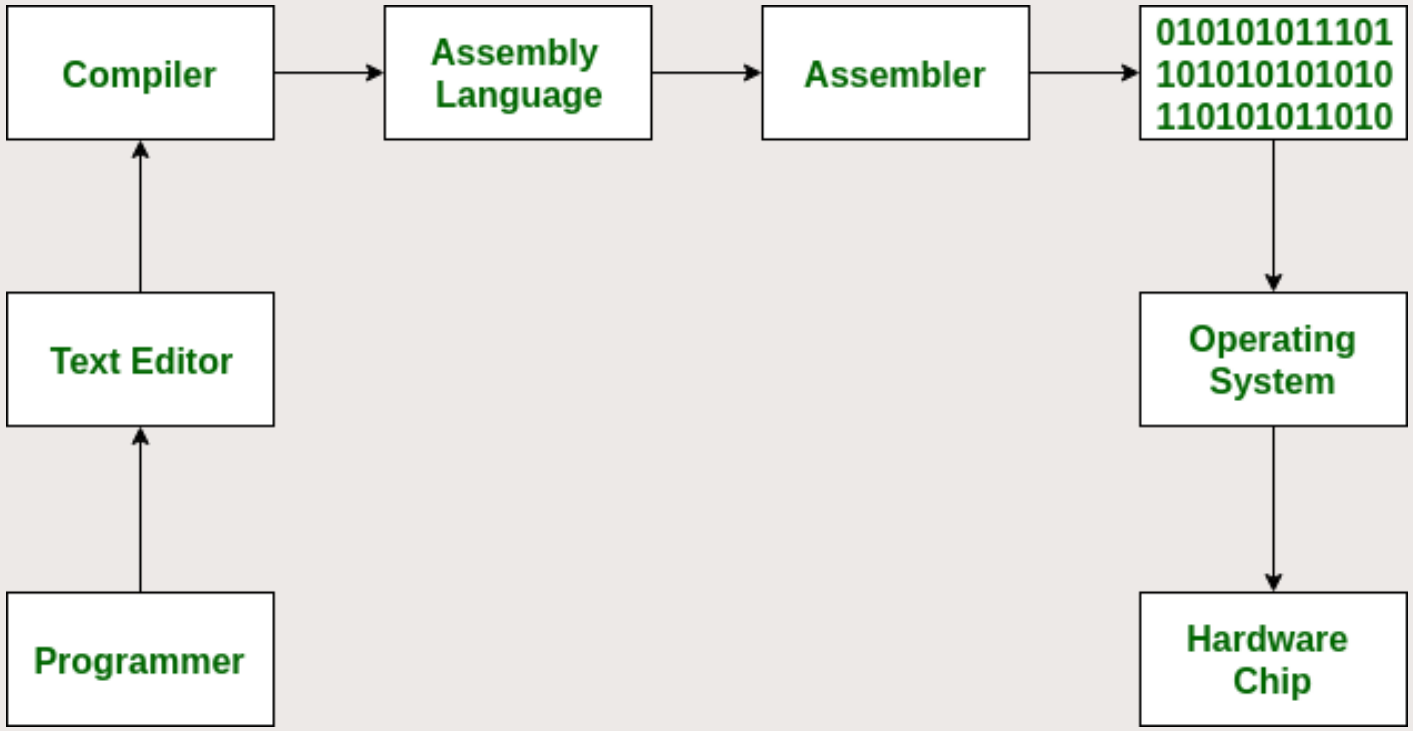
1. CPU performs operations such as **addition, subtraction, comparison**, etc.
2. **RAM** temporarily stores data and programs.
3. **Storage (HDD/SSD)** stores data permanently.

6.Role of Operating System

1. OS acts as an **interface between software and hardware**.
2. It manages CPU, memory and devices such as **keyboard, mouse, display and printer**.

Simple Flow

Programmer → High-Level Code → Compiler → Assembly → Assembler → Machine Code (0s & 1s) → CPU/Hardware → Result



Key Point

The CPU understands only machine-level instructions, while the OS helps programs communicate with and use hardware resources.

Number System

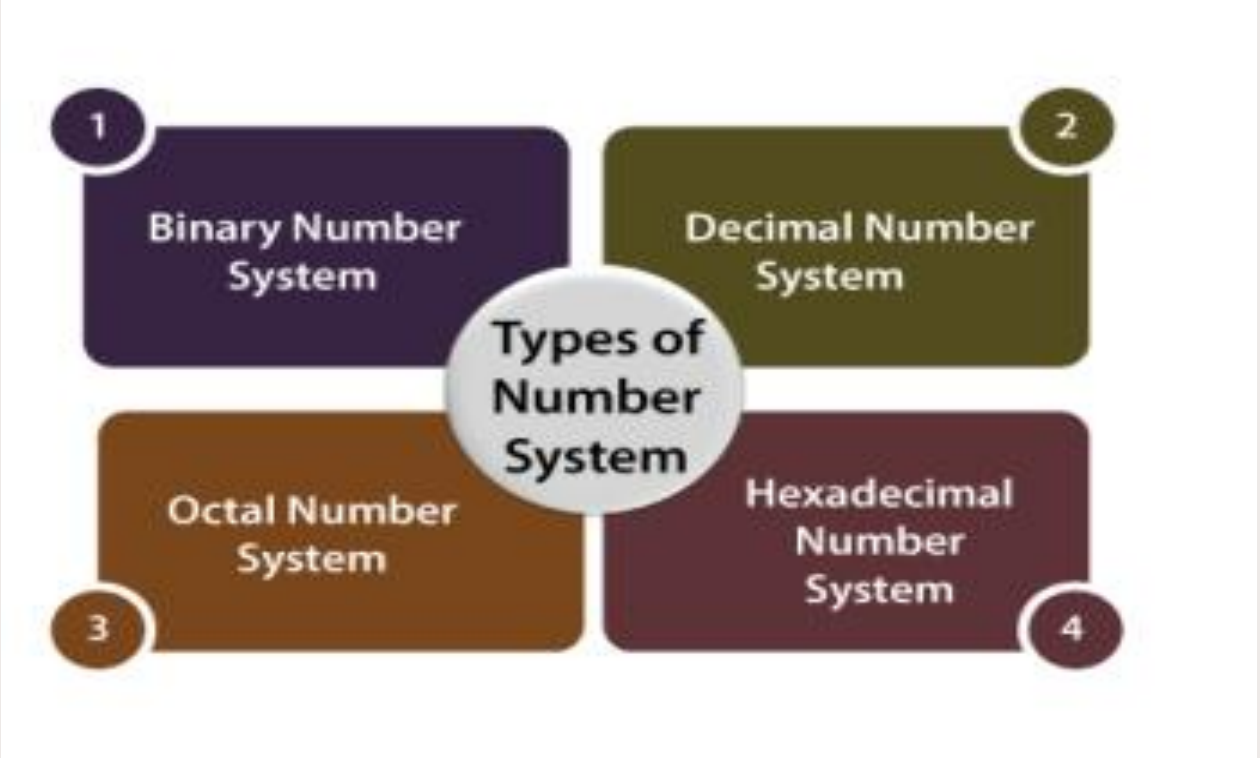
What is a Number System?

A number system is a method of representing numbers using a set of symbols/digits.

In computers, different number systems are used because computers work mainly with binary **(0 and 1)**.

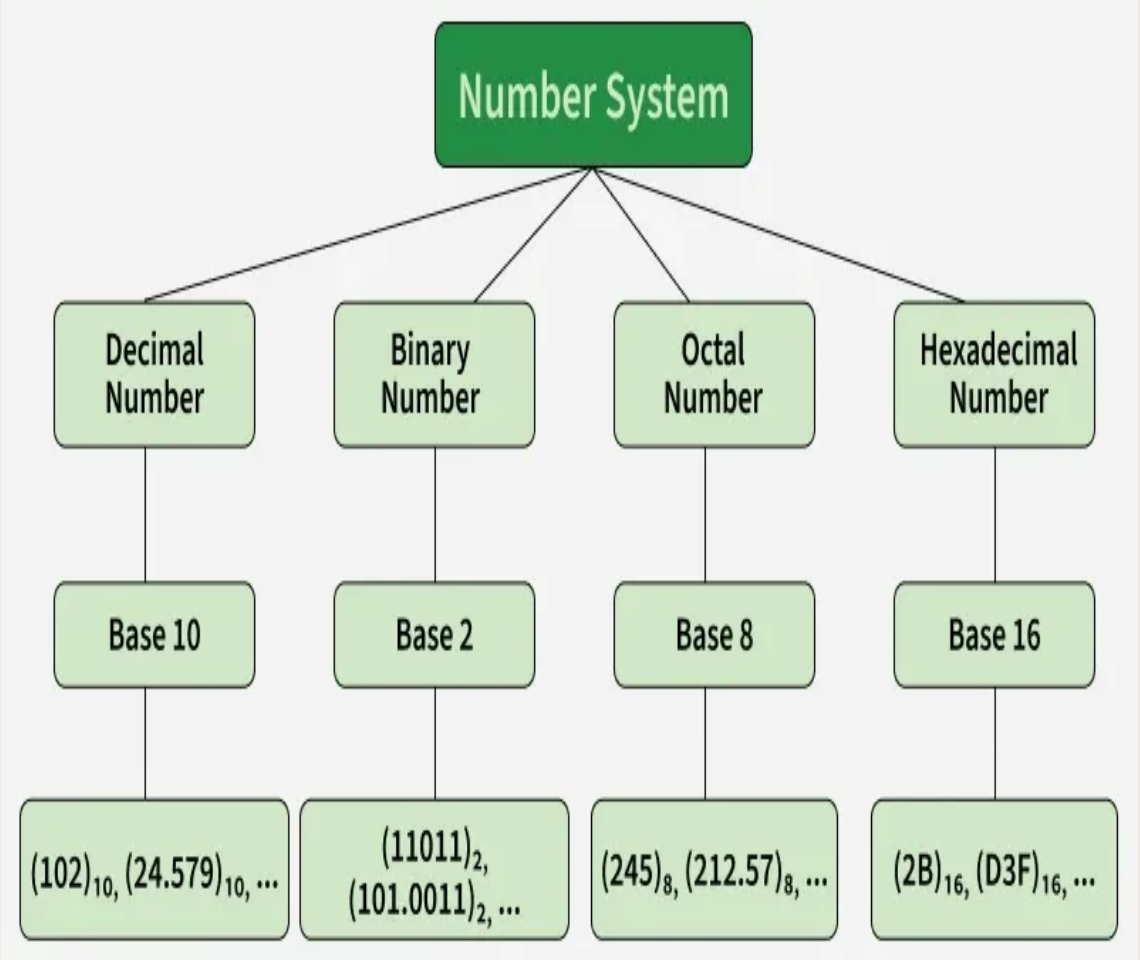
The four important number systems are:

- Decimal – Base 10
- Binary – Base 2
- Octal – Base 8
- Hexadecimal – Base 16



Types of Number System

- **Binary (Base 2):** Uses digits **0** and `1`. Computers use this because electronic circuits have two states: ON (1) or OFF (0).
- **Decimal (Base 10):** Uses digits `0` through `9`. This is the standard number system humans use every day.
- **Octal (Base 8):** Uses digits `0` through `7`. It groups binary numbers into sets of three to make large binary strings shorter.
- **Hexadecimal (Base 16):** Uses digits `0` through `9` and letters `A` through `F` (where A=10, B=11, C=12, D=13, E=14, F=15). It groups binary numbers into sets of four.



Binary Number system

- A binary number system is used in the digital computers. In this number system, it carries only two digits, either 0 or 1.
- There are two types of electronic pulses present in a binary number system. i. the absence of an electronic pulse representing '0'. ii. the presence of electronic pulse representing '1'.
- Each digit is known as a bit.
- A four-bit collection (1101) is known as a nibble.
- A collection of eight bits (11001010) is known as a byte.

Characteristics:

2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1024	512	256	128	64	32	16	8	4	2	1

1. It holds only two values, i.e., either 0 or 1.
2. It is also known as the base 2 number system.
3. The position of a digit represents the 0 power of the base(2). Example: 2^0
4. The position of the last digit represents the x power of the base(2). Example: 2^x , where x represents the last position, i.e., 1

- From right to left - as powers of 2 i.e. 2^0, 2^1, 2^2, etc).

- Number system has two symbols 0(zero) and 1(one) called **Binary Digits or bits**. Its **base is 2.**

- The weights assigned to bits in this system are powers of 2, namely 2^0 , 2^1 , 2^2 ……



Characteristics:

1. It holds only two values, i.e., either 0 or 1.
2. It is also known as the base 2 number system.
3. The position of a digit represents the 0 power of the base(2). Example: 2^0
4. The position of the last digit represents the x power of the base(2). Example: 2^x , where x represents the last position, i.e., 1

- From right to left - as powers of 2 i.e. 2^0, 2^1, 2^2, etc).

Decimal Number System

- The decimal number system contains ten digits from 0 to 9(base 10).
- Here, the successive place value or position, left to the decimal point holds units, tens, hundreds, thousands, and so on.
- The position in the decimal number system specifies the power of the base (10). The 0 is the minimum value of the digit, and 9 is the maximum value of the digit.

10^5	10^4	10^3	10^2	10^1	10^0

Decimal	Binary
	8421
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

For example, the decimal number 2541 consist of the digit 1 in the unit position, 4 in the tens position, 5 in the hundreds position, and 2 in the thousand positions and the value will be written as:

$(2\times10^3) + (5\times10^2) + (4\times10^1) + (1\times10^0)$

$2000 + 500 + 40 + 1$

254

3. Octal Number System

- The octal number system has base 8(means it has only eight digits from 0 to 7).
- With the help of only three bits, an octal number is represented. Each set of bits has a distinct value between 0 and 7
- Octal number system is useful for representing file permissions in Unix/Linux operating systems
- Each digit in the number is at a specific place value that is powers of 8.
- From right to left - as powers of 8 i.e. 8^0, 8^1, 8^2, etc

Octal Number	Binary Equivalent
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

For example:

22_8, 13_8, 17_8, etc, are octal numbers.

Characteristics:

1. An octal number system carries eight digits starting from 0, 1, 2, 3, 4, 5, 6, and 7.
2. It is also known as the base 8 number system.
3. The position of a digit represents the 0 power of the base(8). Example: 8^0
4. The position of the last digit represents the x power of the base(8).

Example: 8^x , where x represents the last position, i.e., 1

4. Hexadecimal Number System

- The number system has a base of 16 means there are total 16 symbols(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) used for representing a number.
- The single-bit representation of decimal values 10, 11, 12, 13, 14, and 15 are represented by A, B, C, D, E, and F.
- Only 4 bits are required for representing a number in a hexadecimal number.
- Each set of bits has a distinct value between 0 and 15.

Binary Number	Hexadecimal Number
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Examples:

$(FAC2)_{16}$, $(564)_{16}$, $(0ABD5)_{16}$, $(1123)_{16}$, $(11F3)_{16}$.

Characteristics:

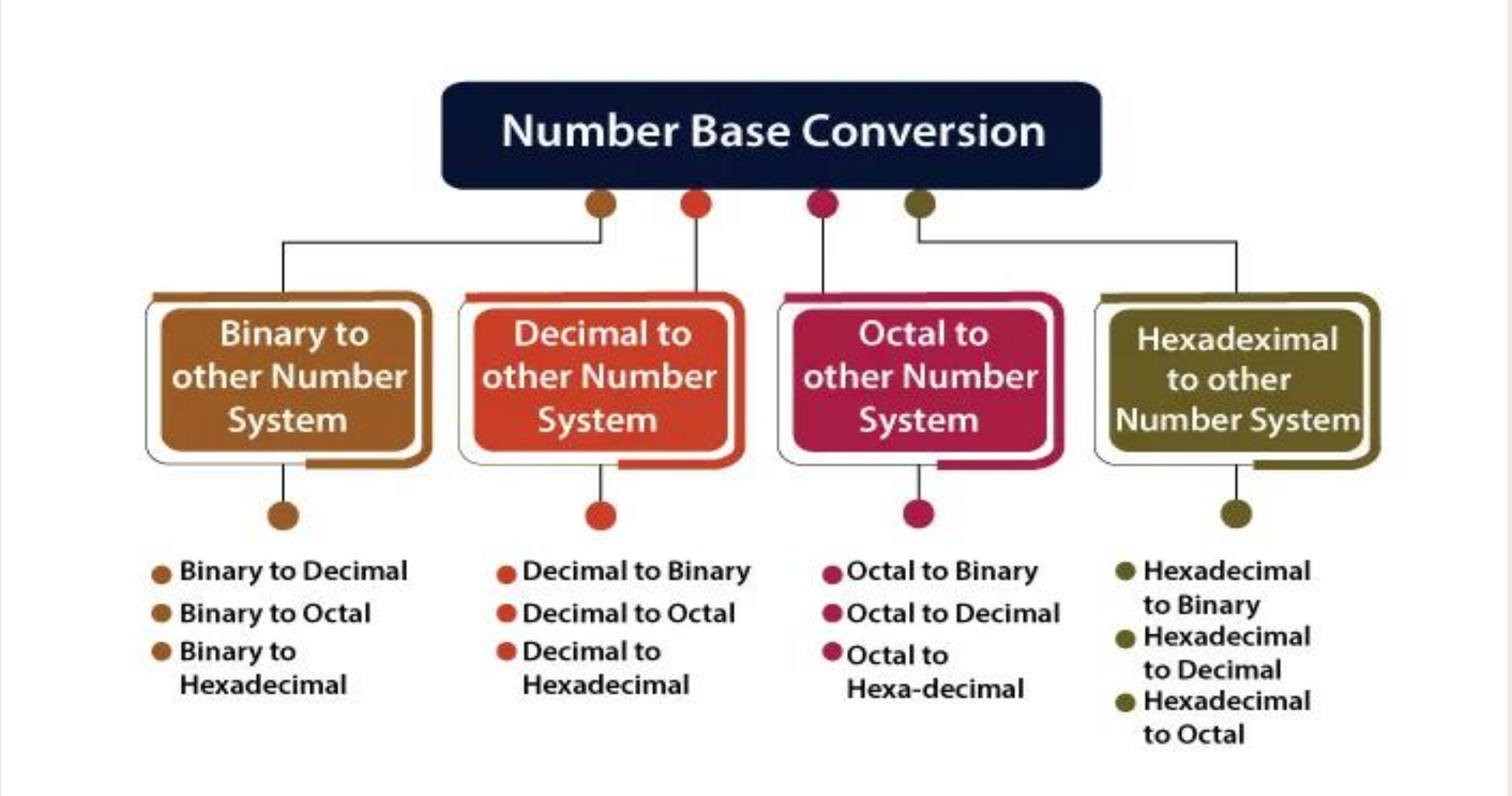
1. It has ten digits from 0 to 9 and 6 letters from A to F.
2. The letters from A to F defines numbers from 10 to 15.
3. It is also known as the base 16number system.
4. In hexadecimal number, the position of a digit represents the 0 power of the base(16).

Example: 16^0

5. In hexadecimal number, the position of the last digit represents the x power of the base(16).

Example: 16^x , where x represents the last position, i.e., 1

Number System Conversions



Number System Conversion Table

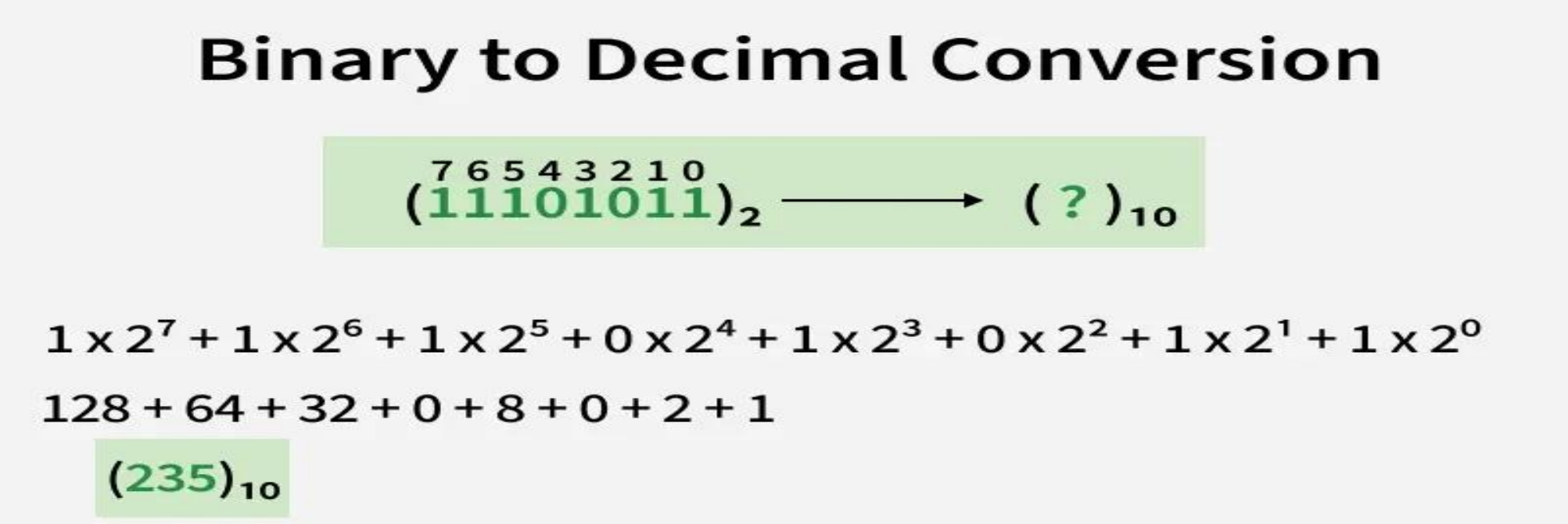
Binary Numbers	Octal Numbers	Decimal Numbers	Hexadecimal Numbers
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F

Binary to other number system conversion

1.Binary to Decimal Conversion

A binary number is converted into a decimal number by multiplying each digit of the binary numbers 1 or 0 to the corresponding to the power of 2 according to the place value.

Example: To convert $(11101011)_2$ into a decimal number.



- **Step 1:** Multiply each digit of the Binary number with the place value of that digit, starting from right to left i.e. from LSB to MSB.
- **Step 2:** Add the result of this multiplication and the decimal number will be formed.

Example: Convert $(1101)_2$ into a decimal number.

Solution: Given a binary number $(1101)_2$.

Now, multiplying each digit from MSB to LSB with reducing the power of the base number 2.

$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

$= 8 + 4 + 0 + 1$

$= 13$

Therefore, $(1101)_2 = (13)_{10}$

Example: Convert the binary number 1001 to a decimal number.

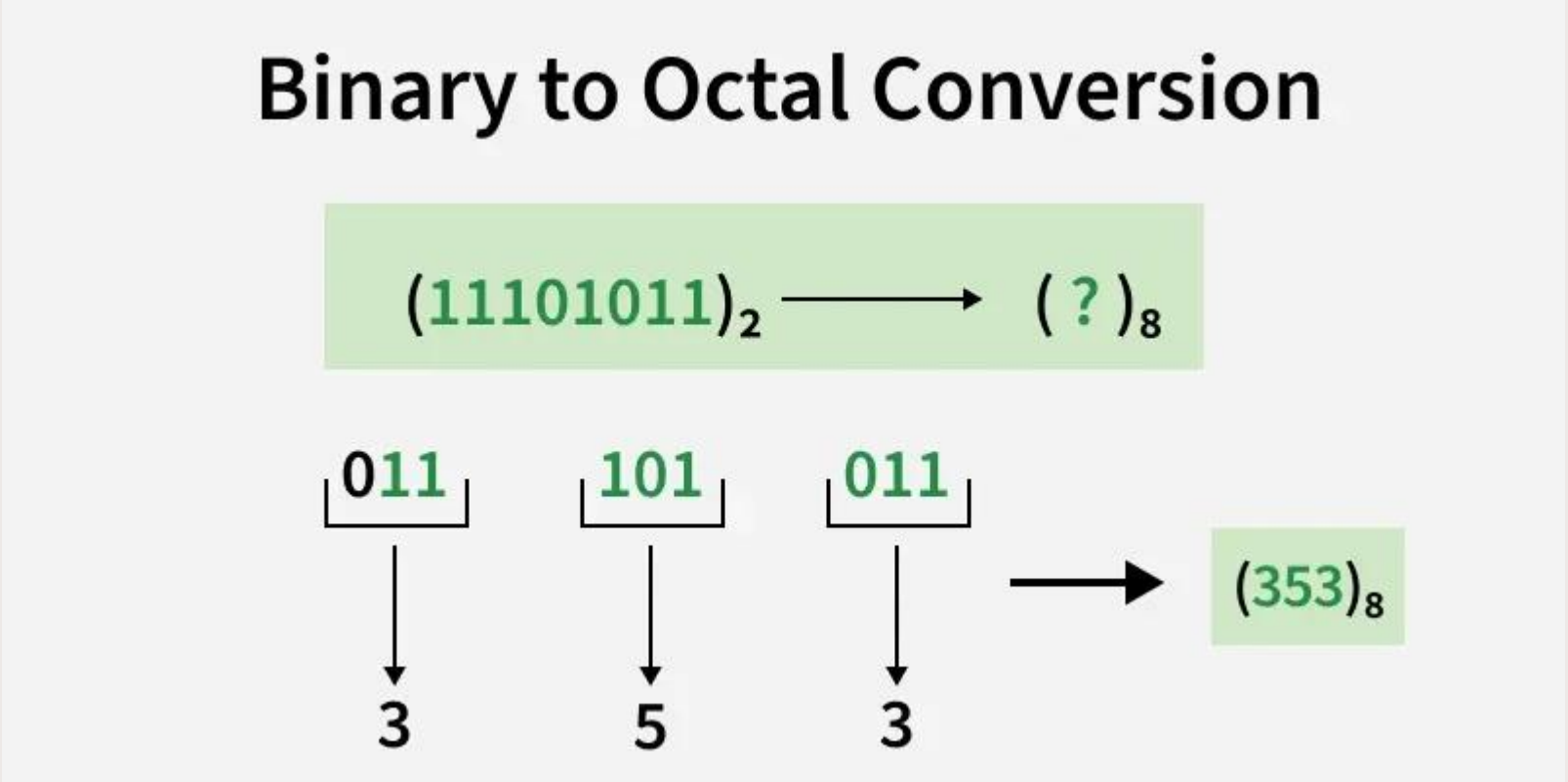
Example: Convert 1101001_2 into an equivalent decimal number.

Example: Convert $(11110111)_2$ into base-10 number system.

2. Binary to Octal Conversion

Binary numbers have a base of 2, while octal numbers have a base of 8. To convert a binary number to an octal number, the base is changed from 2 to 8.

Example: To convert $(11101011)_2$ into an octal number.



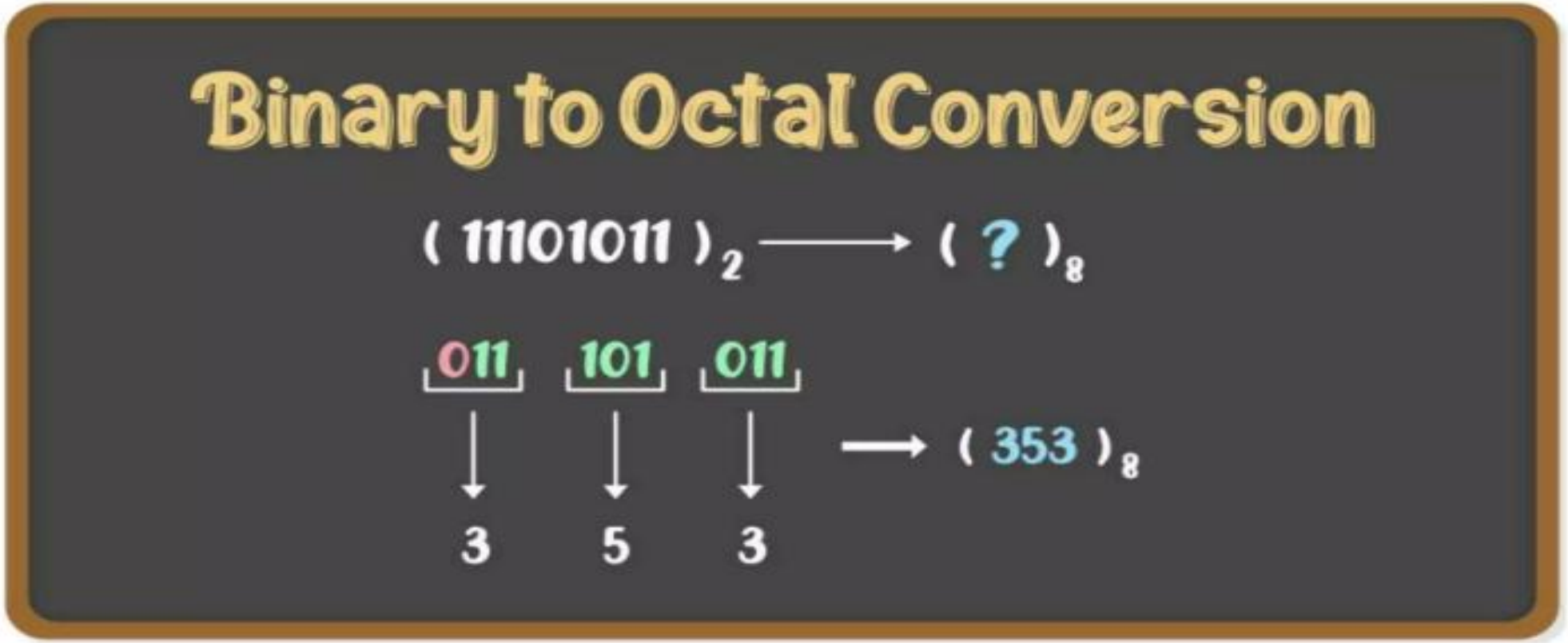
Step 1 − Divide the binary digits into groups of three (starting from the right).
Step 2 − Convert each group of three binary digits to one octal digit.

Example: Binary Number : 10101_2
Calculating Octal Equivalent −

Step	Binary Number	Octal Number
Step 1	10101_2	010 101
Step 2	10101_2	2_8 5_8
Step 3	10101_2	25_8

Binary Number : 10101_2 = Octal Number : 25_8

Example: To convert $(11101011)_2$ into an octal number



Example 1: Convert $(1011)_2 = (?)_8$

Solution:

We will use direct method for binary to octal conversion.

$= (1011)_2$

$= \underline{001}\ \underline{011}$

$= 1\ 3$

$= (1011)_2 = (13)_8$

Example 2: Convert $(0010011)_2 = (____)_8$

Q1. Convert $(1111110)_2$ to $(?)_8$

Q2. Convert $(001100101)_2$ to $(?)_8$

3. Binary to Hexadecimal

Step 1 − Divide the binary digits into groups of four (starting from the right).

Step 2 − Convert each group of four binary digits to one hexadecimal symbol.

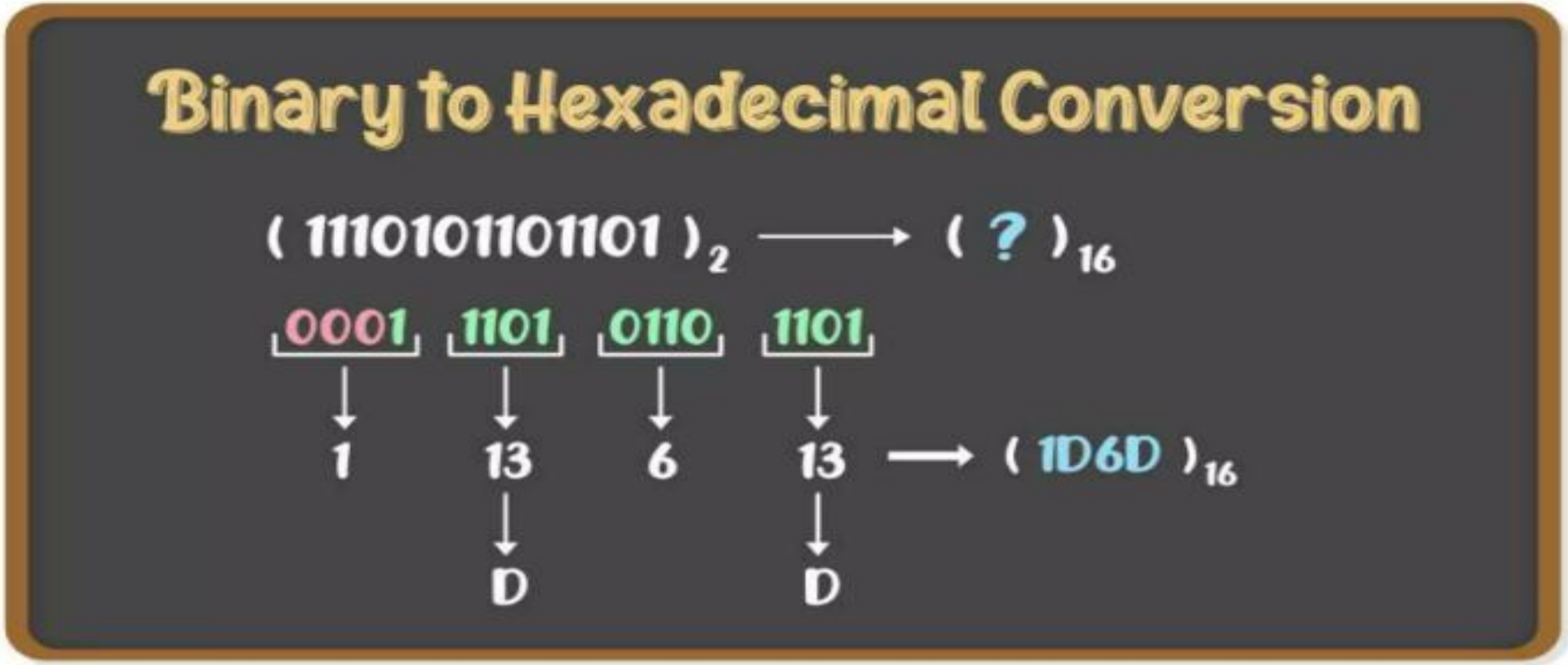
Example: Binary Number : 10101_2

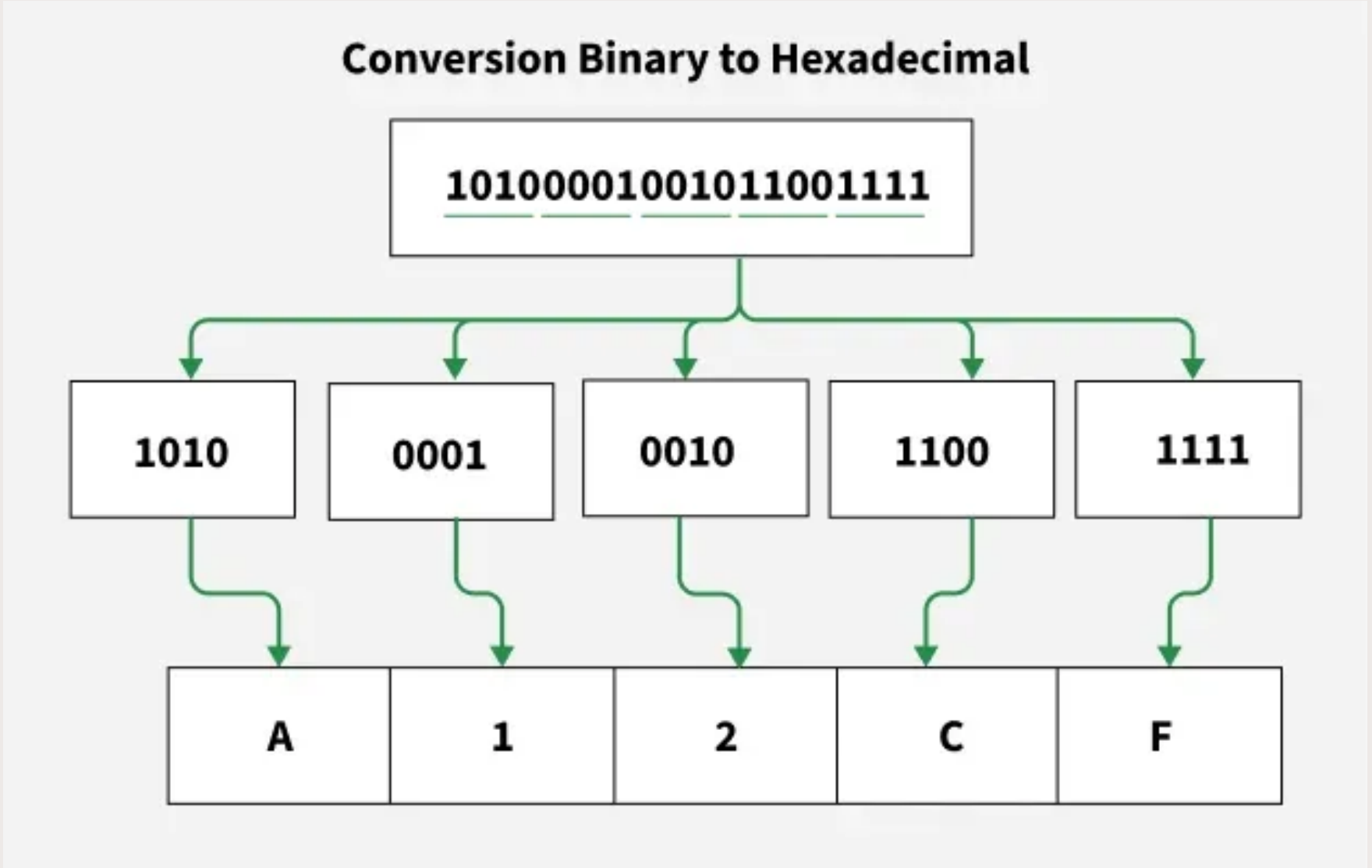
Calculating hexadecimal Equivalent −

Step	Binary Number	Hexadecimal Number
Step 1	10101_2	0001 0101
Step 2	10101_2	1_{10} 5_{10}
Step 3	10101_2	15_{16}

Binary Number : 10101_2 = Hexadecimal Number : 15_{16}

Example: To convert $(1110101101101)_2$ into a hex number




Conversion Binary to Hexadecimal
10100001001011001111
1010
0001
0010
1100
1111
A
1
2
C
F

Question 1: Convert: $(111111101)_2$ = $(_____)_{16}$.

Question 2: Convert: $(01011110001)_2$ = $(_____)_{16}$.

Decimal to other number system conversion

1.Decimal to Binary Conversion

Step 1 — Divide the decimal number to be converted by the value of the new base.
Step 2 — Get the remainder from Step 1 as the rightmost digit (least significant digit) of the new base number.
Step 3 — Divide the quotient of the previous divide by the new base.
Step 4 — Record the remainder from Step 3 as the next digit (to the left) of the new base number.

Repeat Steps 3 and 4, getting remainders from right to left, until the quotient becomes zero in Step 3.
The last remainder thus obtained will be the Most Significant Digit (MSD) of the new base number.

Example: Decimal Number: 29_{10}
Calculating Decimal Equivalent –

- **As mentioned in Steps 2 and 4, the remainders have to be arranged in the reverse order so that the first remainder becomes the Least Significant Digit (LSD) and the last remainder becomes the Most Significant Digit (MSD).**
- **Decimal Number : 29_{10} = Binary Number : $11101_{2.}$**

Step	Operation	Result	Remainder
Step 1	29 / 2	14	1
Step 2	14 / 2	7	0
Step 3	7 / 2	3	1
Step 4	3 / 2	1	1
Step 5	1 / 2	0	1

Decimal to Binary Conversion

$(243)_{10} \longrightarrow (?)_2$



2	243	1
2	121	1
2	60	0
2	30	0
2	15	1
2	7	1
2	3	1
	1	

$\longrightarrow (11110011)_2$

Example 3: Convert $(50)_{10}$ to Binary

Division	Quotient	Remainder
50 ÷ 2	25	0
25 ÷ 2	12	1
12 ÷ 2	6	0
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Therefore:
$(50)_{10} = (110010)_{2}$

Example 1: Convert $(25)_{10}$ to Binary

Example 2: Convert $(45)_{10}$ to Binary

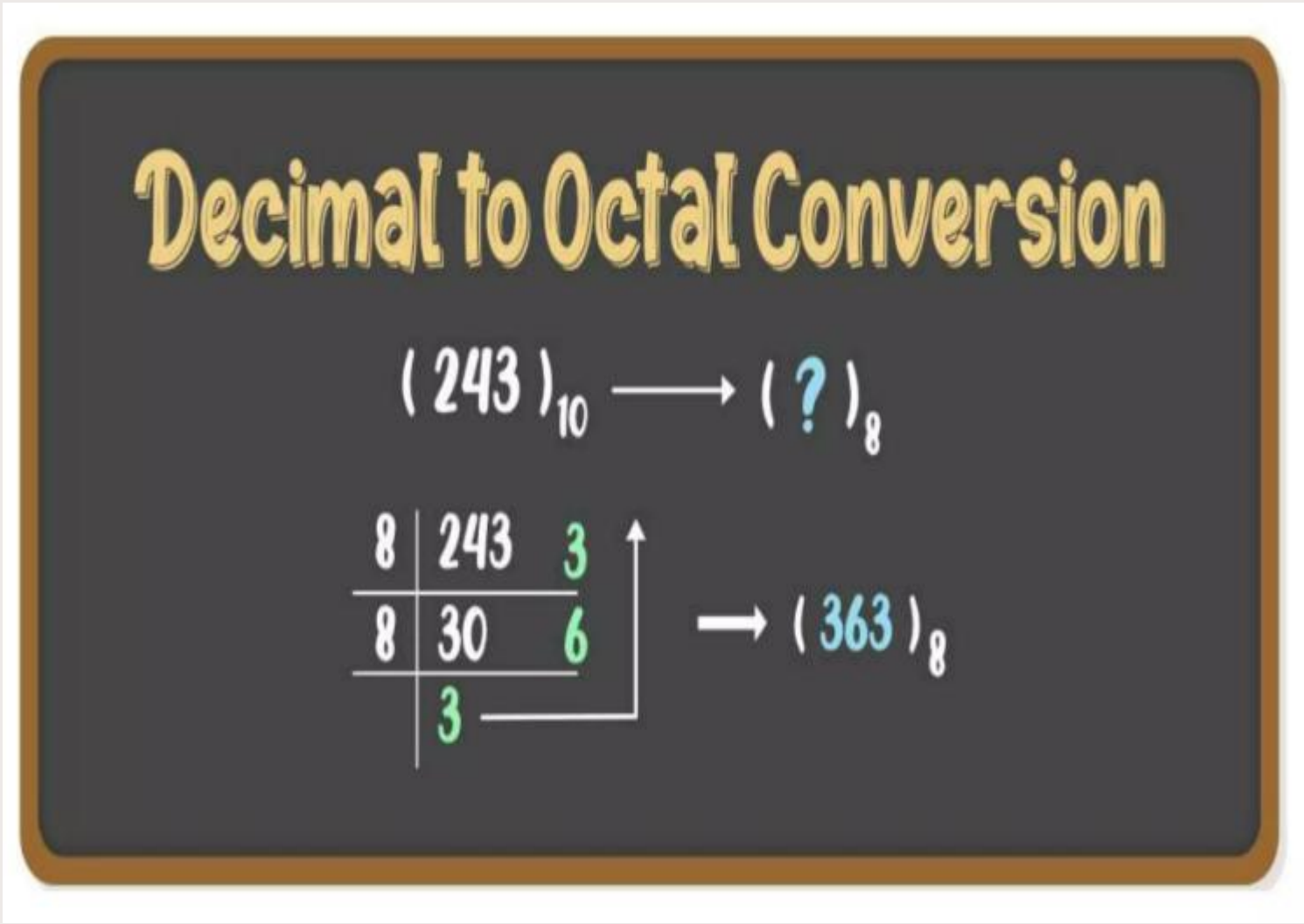
Example 4: Convert $(100)_{10}$ to Binary

Example 4: Convert $(75)_{10}$ to Binary

2.Decimal to Octal Conversion

- **Step 1:** Divide the Decimal Number with the base of the number system to be converted to. Here the conversion is to octal, hence the divisor will be 8.
- **Step 2:** The remainder obtained from the division will become the least significant digit of the new number.
- **Step 3:** The quotient obtained from the division will become the next dividend and will be divided by base i.e. 8.
- **Step 4:** The remainder obtained will become the second least significant digit i.e. it will be added in the left of the previously obtained digit.

Now, the steps 3 and 4 are repeated until the quotient obtained becomes 0, and the remainders obtained after each iteration are added to the left of the existing digits.


Decimal to Octal Conversion
(243)10 ⟶ (?)8
8 | 243 | 3
8 | 30 | 6
3
⟶ (363)8

Example 1: Convert $(25)_{10}$ to Octal

Division	Quotient	Remainder
25 ÷ 8	3	1
3 ÷ 8	0	3

Read remainders from bottom to top:
$(25)_{10} = (31)_{8}$

Example 1: Convert $(50)_{10}$ to Octal

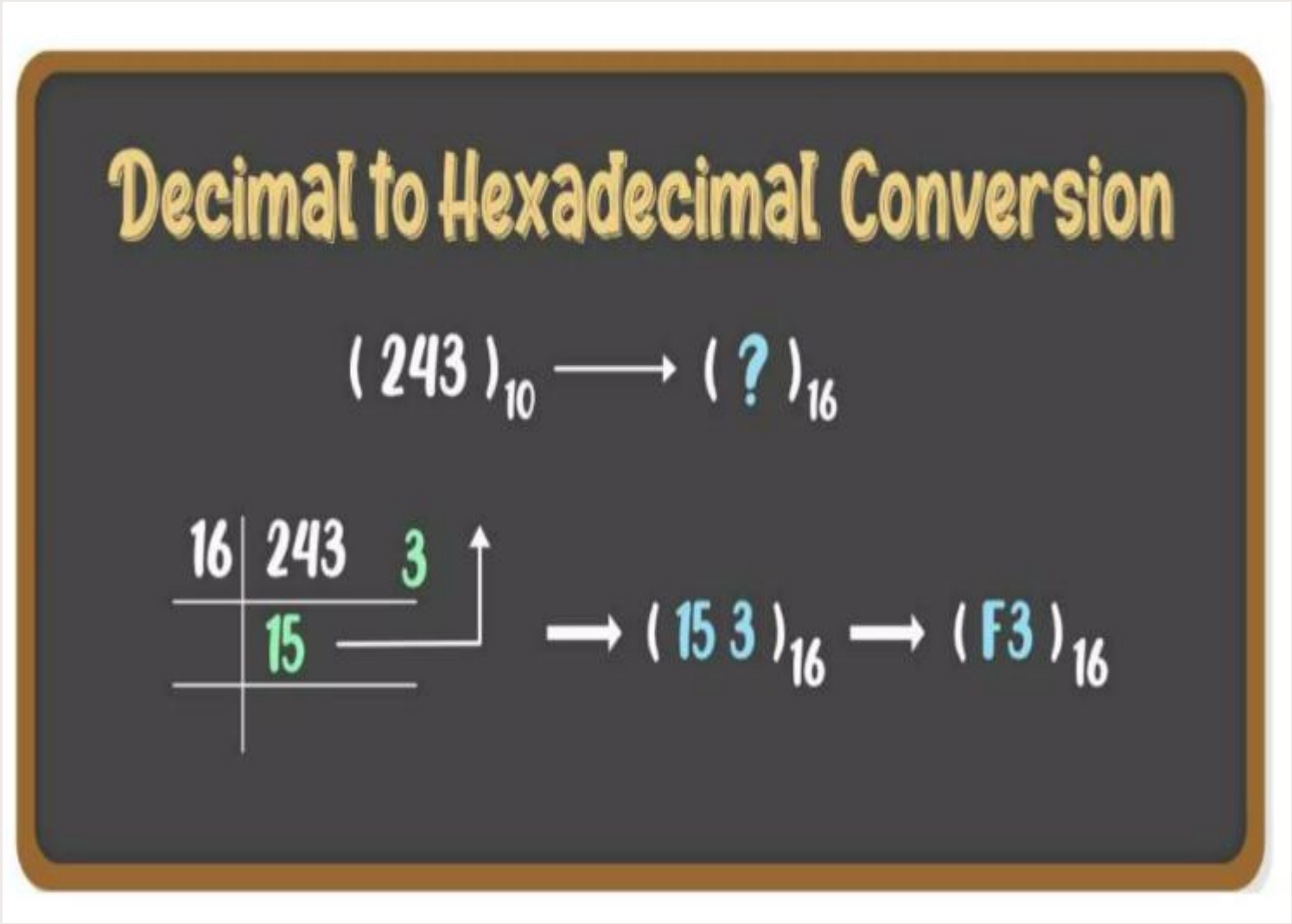
Example 2: Convert $(125)_{10}$ to Octal

Example 3: Convert $(100)_{10}$ to Octal

3 .Decimal to Hexadecimal Conversion

- **Step 1:** Divide the Decimal Number with the base of the number system to be converted to. Here the conversion is to Hex hence the divisor will be 16.
- **Step 2:** The remainder obtained from the division will become the least significant digit of the new number.
- **Step 3:** The quotient obtained from the division will become the next dividend and will be divided by base i.e. 16.
- **Step 4:** The remainder obtained will become the second least significant digit i.e. it will be added in the left of the previously obtained digit.

Now, the steps 3 and 4 are repeated until the quotient obtained becomes 0, and the remainders obtained after each iteration are added to the left of the existing digits.


Decimal to Hexadecimal Conversion
(243)10 ⟶ (?)16
16 | 243 3
15
⟶ (15 3)16 ⟶ (F3)16

Example 1: Convert $(25)_{10}$ to Hexadecimal

Division	Quotient	Remainder
25 ÷ 16	1	9
1 ÷ 16	0	1

Therefore:
$(25)_{10} = (19)_{16}$

Example 2: Convert $(100)_{10}$ to Hexadecimal

Example 3: Convert $(255)_{10}$ to Hexadecimal

Example 4: Convert $(200)_{10}$ to Hexadecimal

Easy Rule to Remember

Conversion	Divide by	Remainder
Decimal → Binary	2	0 or 1
Decimal → Octal	8	0 to 7
Decimal → Hexadecimal	16	0 to 15

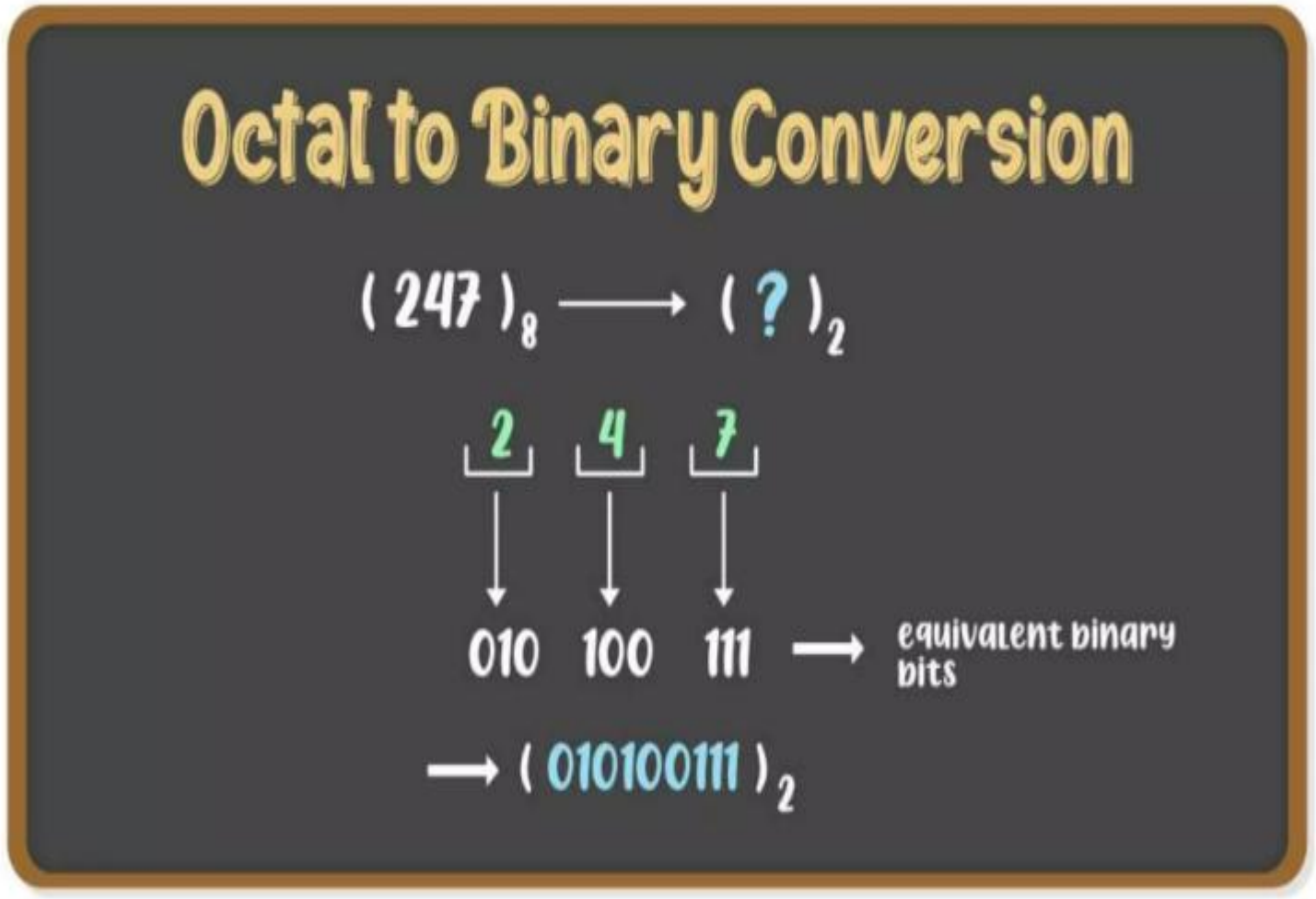
Octal to other number system Conversion

Octal to Binary Conversion

Octal numbers are represented in base 8, but the binary numbers are of base 2. Hence, to convert an octal number to a binary number, the base of that number is to be changed. Follow the steps given below:

- **Step 1:** Write each digit of the octal number separately.
- **Step 2:** Convert each digit into an equivalent group of three binary digits.
- **Step 3:** Combine these groups to form the whole binary number.

Example: $(247)_8$ is to be converted to binary



Example 1: Convert $(25)_8$ to Binary

Now combine the binary values:

Step	Octal Digit	3-bit Binary
1	2	010
2	5	101

Octal Number	Binary Conversion
25_8	$010\ 101_2$

Therefore:
$(25)_8 = (010101)_2$

After removing the unnecessary leading zero:
$(25)_8 = (10101)_2$

Example 2: Convert $(347)_8$ to Binary

Example 3: Convert $(562)_8$ to Binary

Example 4: Convert $(703)_8$ to Binary

Example 5: Convert $(1256)_8$ to Binary

Octal to Decimal Conversion

Here are the steps to convert a number from octal to decimal:

- Step 1: Since an octal number only uses digits from 0 to 7, we first arrange the octal number with the power of 8.
- Step 2: We evaluate all the power of 8 values such as 8^0 is 1, 8^1 is 8, etc., and write down the value of each octal number.
- Step 3: Once the value is obtained, we multiply each number.
- Step 4: Final step is to add the product of all the numbers to obtain the decimal number.

Power	Value
8^0	1
8^1	8
8^2	64
8^3	512
8^4	4096

Let us look at an example, convert $(140)_8$ into a decimal number.

Step 1: Write 140 with the power of 8. Start from the right-hand side.

$1 \times 8^2 + 4 \times 8^1 + 0 \times 8^0$

Step 2: Evaluate the power of 8 values for each octal number.

$8^2 = 64$, $8^1 = 8$, $8^0 = 1$

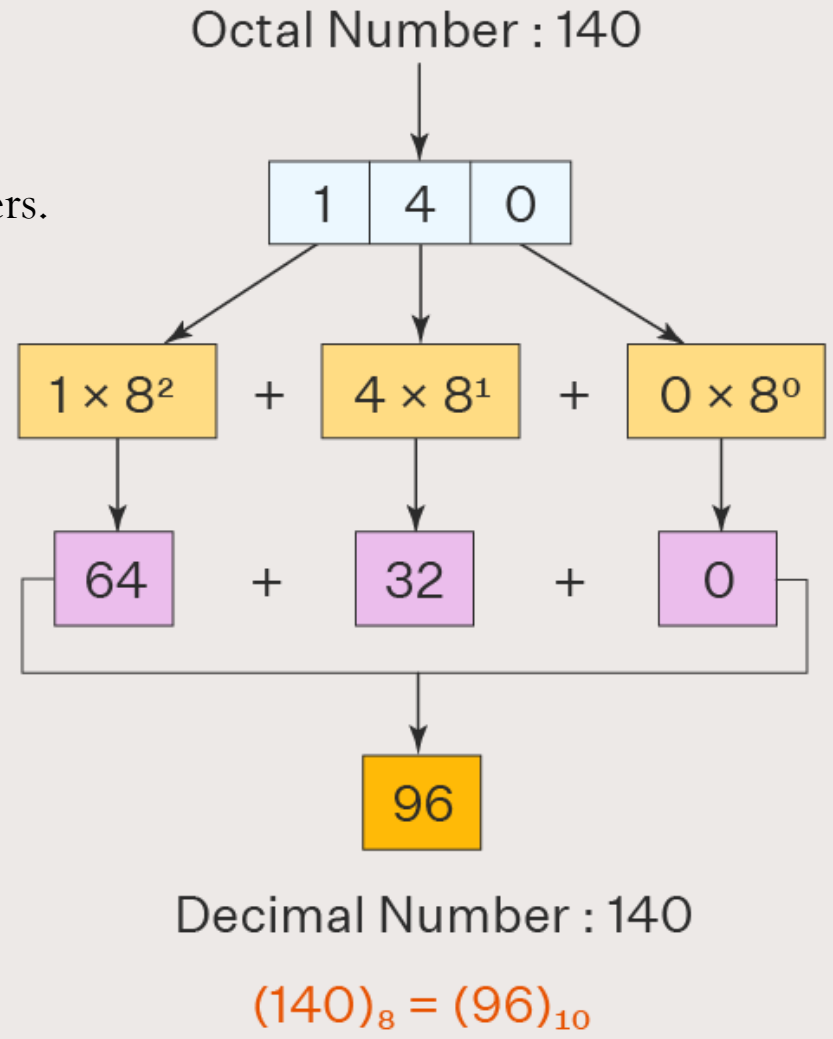
Step 3: Multiply each of the power of 8 numbers with the respective numbers.

$1 \times 64 + 4 \times 8 + 0 \times 1 = 64 + 32 + 0$

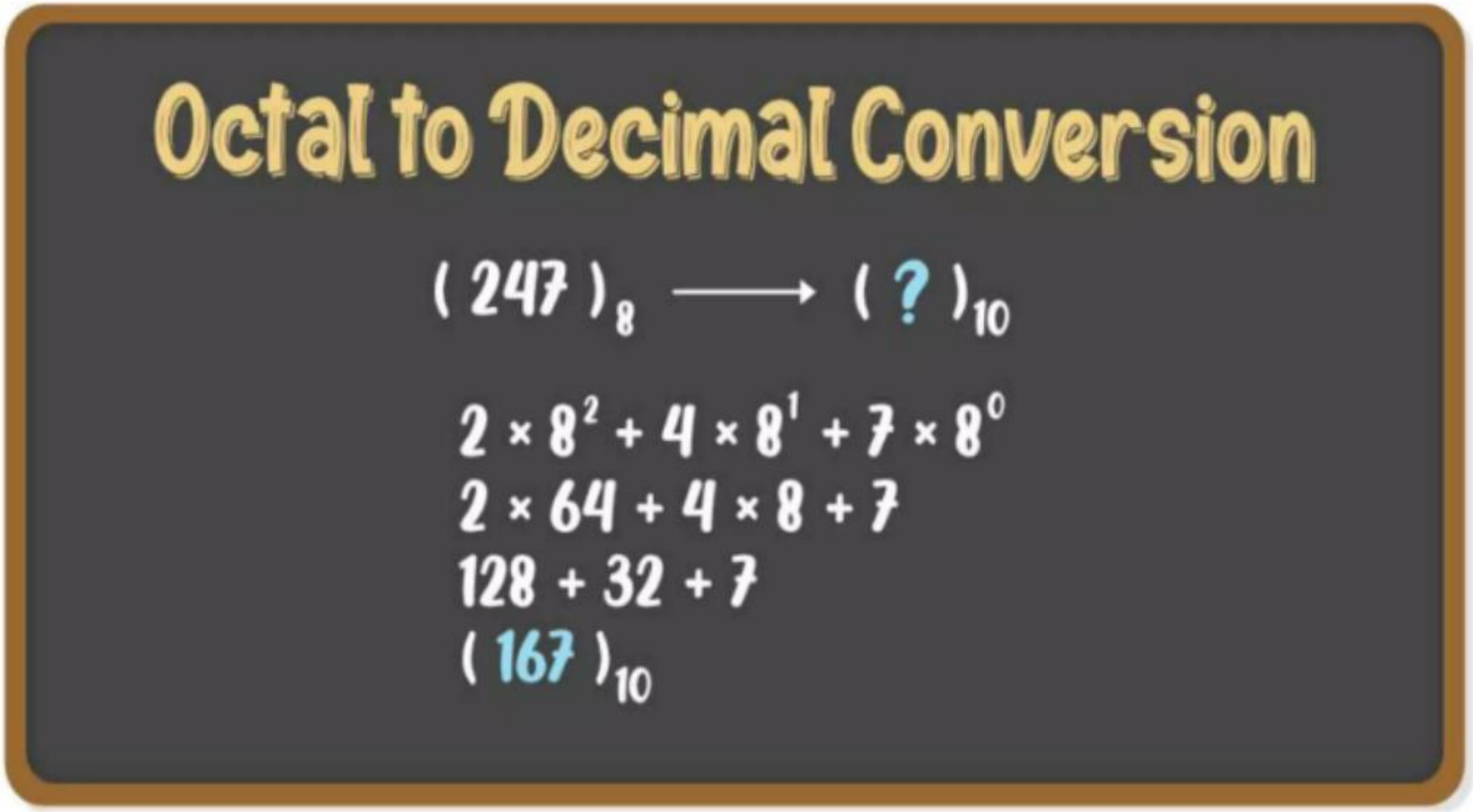
Step 4: Add the values to obtain the decimal number.

$64 + 32 + 0 = 96$.

Therefore, $(140)_8 = (96)_{10}$.



Example :


Octal to Decimal Conversion
$(247)_8 \longrightarrow (?)_{10}$
$2 \times 8^2 + 4 \times 8^1 + 7 \times 8^0$
$2 \times 64 + 4 \times 8 + 7$
$128 + 32 + 7$
$(167)_{10}$

Example 1: Convert $(25)_8$ to Decimal

Example 2: Convert $(347)_8$ to Decimal

Example 3: Convert $(562)_8$ to Decimal

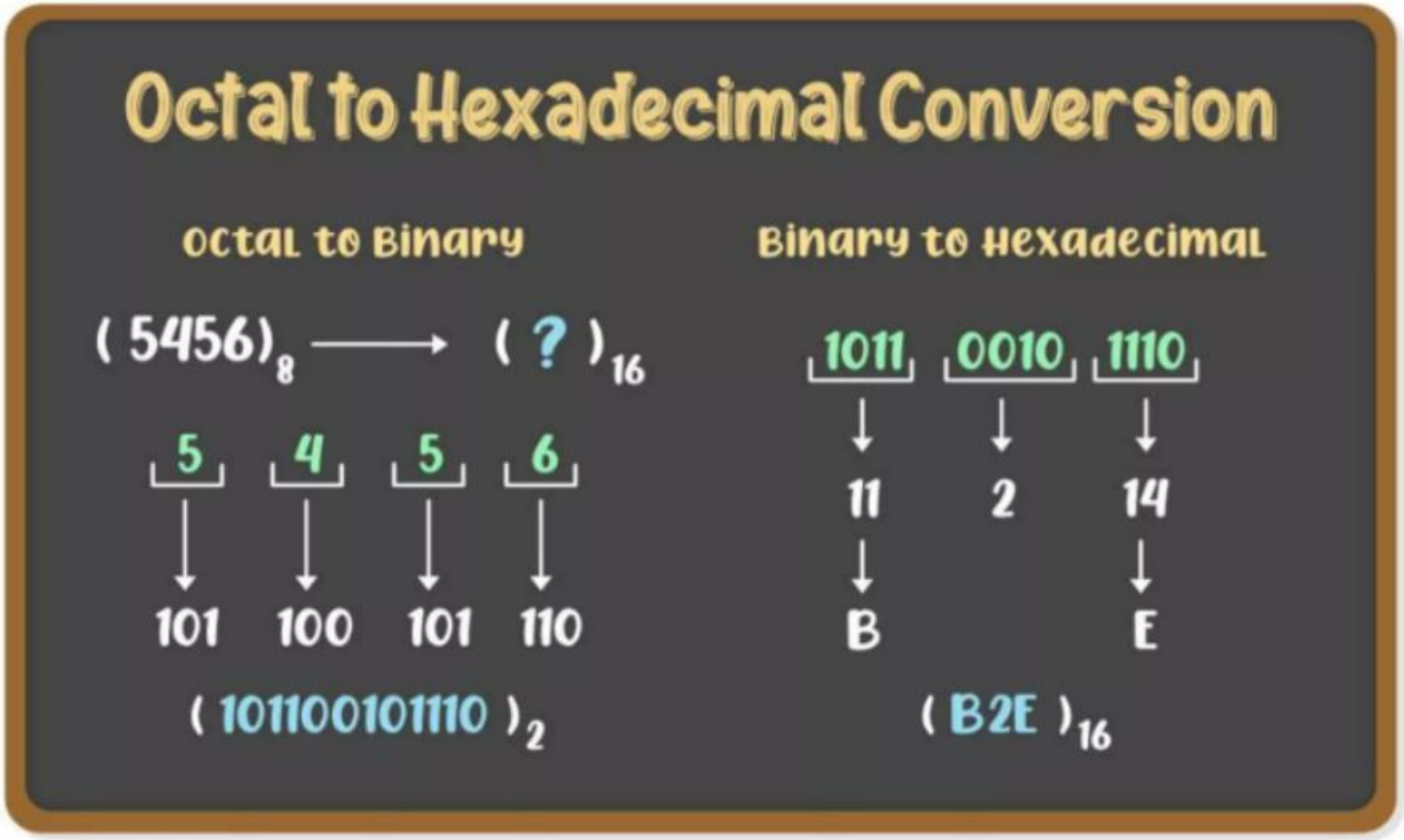
Example 4: Convert $(703)_8$ to Decimal

3. Octal to Hexadecimal Conversion

Octal numbers are represented in base 8, but the hexadecimal numbers are of base 16. Hence, to convert an octal number to a hex number, the base of that number is to be changed. Follow the steps given here :

- **Step 1:** We need to convert the Octal number to Binary first. For that, follow the steps given in the above conversion.
- **Step 2:** Now to convert the binary number to Hex number, divide the binary digits into groups of four digits starting from right to left i.e. from LSB to MSB.
- **Step 3:** Add zeros prior to MSB to make it a proper group of four digits(if required)
- **Step 4:** Now convert these groups into their relevant decimal values.
- **Step 5:** For values from 10-15, convert it into Hex symbols i.e. from A-F

Example: $(5456)_8$ is to be converted to hex



Conversion Tables
Octal → Binary

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Binary → Hexadecimal

Binary	Hexadecimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Example 1: Convert $(25)_8$ to Hexadecimal
Step 1: Octal → Binary

Octal Digit	Binary
2	010
5	101

So:
$(25)_8 = (010101)_2$

Step 2: Group binary into 4 bits from right

010101
↓
01 0101

Add zeros to the left:
0001 0101

Step 3: Binary → Hexadecimal
Therefore:
$(25)_8 = (15)_{16}$

Binary	Hexadecimal
0001	1
0101	5

Example 2: Convert $(347)_8$ to Hexadecimal

Step 1: Octal → Binary
Therefore:
$(347)_8 = (011100111)_2$

Octal Digit	Binary
3	011
4	100
7	111

Step 2: Group into 4 bits
011100111
↓
0000 1110 0111

Step 3: Convert to hexadecimal

Binary	Hexadecimal
0000	0
1110	E
0111	7

Ignore the leading zero:
$(347)_8 = (E7)_{16}$

Example 3: Convert $(562)_8$ to Hexadecimal

Example 4: Convert $(703)_8$ to Hexadecimal

Hexadecimal Number System

The hexadecimal system is a base-16 number system that plays an important role in computing and digital systems. It uses sixteen symbols to represent values:

Digits (0 to 9) and the letters A to F, where A = 10, B = 11, and so on up to F = 15.

Decimal	Binary	Hexa-Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Place Value of Digits

The numbers in the hexadecimal number system have weightage in powers of 16. The power of 16 increases as the digit is shifted towards the left of the number. This is explained by the example as,

Example: $(AB12)_{16}$

Place value of each digit in $(AB12)_{16}$ *is,*

$= A \times 16^3 + B \times 16^2 + 1 \times 16^1 + 2 \times 16^0$

Hex to Decimal Conversion

The hexadecimal number system, commonly known as base-16 or just hex, is a system of numbers that has 16 different symbols for representing various values.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

*Here: **A** = 10, **B** = 11, **C** = 12, **D** = 13, **E** = 14, **F** = 15.*

Steps to convert Hexadecimal to Decimal

Using the base number 16, the hexadecimal to decimal conversion is performed.

Step 1: *From the table previously mentioned, write the number's hexadecimal equivalent in decimal form for each digit.*

Step 2: *Beginning with the rightmost digit, multiply the digits in order from right to left with exponents of 16 i.e., 16^0, 16^1, 16^2, . . .*

Step 3: *Following that, add each product. The decimal number is the sum of the results.*

Example: Convert 6CF (hex) to decimal.

Solution:

6CF is the given hexadecimal number. In hexadecimal number system

- *6 = 6*
- *C = 12*
- *F = 15*

Start at the unit's place of the number and multiply each digit by a power of 16 in order to convert this to a decimal number system.

$6CF = (6 \times 16^2) + (12 \times 16^1) + (15 \times 16^0)$
$\Rightarrow 6CF = (6 \times 256 + 12 \times 16 + 15 \times 1)$
$\Rightarrow 6CF = 1536 + 192 + 15$
$\Rightarrow 6CF = 1743$

Thus, decimal value of 6CF is 1743.

Q 1 Convert (4C7) to a decimal number.

Q 2 Convert (16F) into its equivalent Decimal number**.**

Q.3. Convert 5BC (hex) to decimal.

How to Convert Hexadecimal to Binary Number?

To convert a hexadecimal number into its equivalent binary number, follow the steps given here:

- **Step 1:** Take given hexadecimal number
- **Step 2:** Find the number of digits in the decimal
- **Step 3:** If it has n digits, multiply each digit with 16^{n-1} where the digit is in the nth position
- **Step 4:** Add the terms after multiplication
- **Step 5:** The result is the decimal number equivalent to the given hexadecimal number. Now we have to convert this decimal to binary number.
- **Step 6:** Divide the decimal number with 2
- **Step 7:** Note the remainder
- **Step 8:** Do the above 2 steps for the quotient till the quotient is zero
- **Step 9:** Write the remainders in the reverse order.
- **Step 10:** The result is the required binary number.

Hexa To Binary Examples

Question 1: Convert $A2B_{16}$ to an equivalent binary number.

Solution: Given hexadecimal number = $A2B_{16}$

First, convert the given hexadecimal to the equivalent decimal number.

$A2B_{16} = (A \times 16^2) + (2 \times 16^1) + (B \times 16^0)$

$= (A \times 256) + (2 \times 16) + (B \times 1)$

$= (10 \times 256) + 32 + 11$

$= 2560 + 43$

$= 2603$(Decimal number)

Now we have to convert 2603_{10} to binary

2 | 2603
2 | 1301 -- 1
2 | 650 -- 1
2 | 325 -- 0
2 | 162 -- 1
2 | 81 -- 0
2 | 40 -- 1
2 | 20 -- 0
2 | 10 -- 0
2 | 5 -- 0
2 | 2 -- 1
2 | 1 -- 0
2 | 0 -- 1

The binary number obtained is 101000101011_2

Hence, $A2B_{16} = 101000101011_2$

Question 2: Convert E_{16} to an equivalent binary number.

Solution: Given, a hexadecimal number is E.

First, convert the given hexadecimal to the equivalent decimal number.

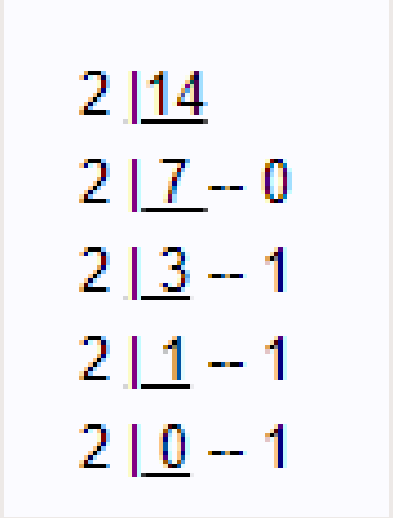
$E_{16} = E \times 16^0$

$= E \times 1$

$= E$

$=14$ (Decimal number)

Now we have to convert 14_{10} to binary number.



The binary number obtained is 1110_2

Hence, $E_{16} = 1110_2$

Question 3: Convert 30_{16} to an equivalent binary number.

Hexadecimal to Octal Conversion

- For each given hexadecimal number digit, write the equivalent binary number. If any of the binary equivalents are less than 4 digits, add 0's to the left side.
- Combine and make the groups of binary digits from right to left, each containing 3 digits. Add 0's to the left if there are less than 3 digits in the last group.
- Find the octal equivalent of each binary group.

Let us see an example here:

Example: Convert $1BC_{16}$ into an octal number.

Solution: Given, $1BC_{16}$ is a hexadecimal number.

1 → 0001, B → 1011, C →1100

Now group them from right to left, each having 3 digits.

000, 110, 111, 100

000→0, 110 →6, 111→7, 100→4

Hence, $1BC_{16} = 674_8$

Q.1: Find the equivalent octal form of $C1_{16}$.
Q.2: Find the equivalent octal form of F_{16}.
Q.3: Find the equivalent octal form of 105_{16}

Q.1: Find the equivalent octal form of $C1_{16}$.

Solution: Given, a hexadecimal number is C1

$C1_{16} = (C \times 16^1) + (1 \times 16^0)$

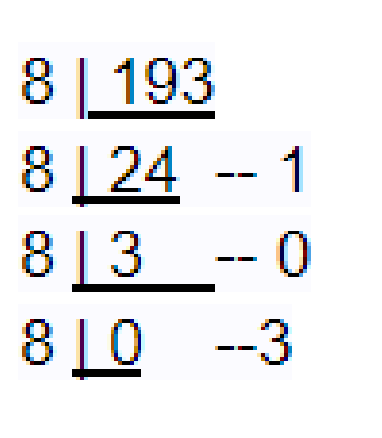
$= C \times 16 + 1 \times 1$

$=12 \times 16 + 1$

$= 192 + 1$

$C1_{16} =193$ (Decimal form)

Now we have to convert this decimal to octal number;



The octal number is 301_8

Hence, $C1_{16} = 301_8$